```
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
)
func main() {
    s := g.Server()
    s.SetIndexFolder(true)
    s.AddStaticPath("/public", "./public")
    s.BindHandler("/", func(r *ghttp.Request) {
        r.Response.RedirectTo("/index")
    })
    s.BindController("/index", new(handlers.IndexHandler))
    s.BindController("/system_status", new(handlers.SystemStatusHandler))
    s.BindController("/voice_interaction", new(handlers.VoiceInteractionHandler))
    s.BindController("/remote_desktop", new(handlers.RemoteDesktopHandler))
    s.BindController("/video_monitoring", new(handlers.VideoMonitoringHandler))
    s.BindController("/hardware_status", new(handlers.HardwareStatusHandler))
    s.BindController("/software_update", new(handlers.SoftwareUpdateHandler))
    s.BindController("/data_sync", new(handlers.DataSyncHandler))
    s.BindController("/alarm_record", new(handlers.AlarmRecordHandler))
    s.BindController("/performance_analysis", new(handlers.PerformanceAnalysisHandler))
    s.BindController("/maintenance_log", new(handlers.MaintenanceLogHandler))
    s.Run()
}
toml
[database]
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
)
type IndexHandler struct{}
func (h *IndexHandler) Index(r *ghttp.Request) {
    r.Response.WriteTpl("index.html")
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
)
type SystemStatusHandler struct{}
func (h *SystemStatusHandler) Index(r *ghttp.Request) {
    status, err := models.GetSystemStatus()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    r.Response.WriteJson(status)
}
```

```
package models
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
)
func GetSystemStatus() (gdb.Result, error) {
    return g.DB().Table("system_status").All()
}
package utils
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/os/gcfg"
    "github.com/gogf/gf/util/gconv"
)
func InitDB() error {
    cfg := gcfg.New()
    if err := cfg.ParseConfigFile("config/config.toml"); err != nil {
        return err
    }
    dbConfig := cfg.GetMap("database")
    for k, v := range dbConfig {
        if err := g.DB(k).SetConfig(gconv.Map(v)); err != nil {
            return err
        }
    }
    return nil
}
package handlers_test
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "github.com/gogf/gf/test/gtest"
    "project/internal/handlers"
    "testing"
)
func TestIndexHandler(t *testing.T) {
    s := g.Server()
    s.BindController("/index", new(handlers.IndexHandler))
    s.SetPort(8199)
    s.Start()
    defer s.Shutdown()
    gtest.C(t, func(t *gtest.T) {
        r, e := ghttp.Get("http://127.0.0.1:8199/index")
        t.AssertNil(e)
        t.Assert(r.StatusCode, 200)
    })
}
package models_test
import (
    "github.com/gogf/gf/test/gtest"
```

```
    "project/internal/models"
    "project/internal/utils"
    "testing"
)
func TestGetSystemStatus(t *testing.T) {
    if err := utils.InitDB(); err != nil {
        t.Fatal(err)
    }
    gtest.C(t, func(t *gtest.T) {
        status, err := models.GetSystemStatus()
        t.AssertNil(err)
        t.AssertNE(status.Len(), 0)
    })
}
package main
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
)
func main() {
    s := g.Server()
    s.BindHandler("/", func(r *ghttp.Request) {
        r.Response.WriteTpl("index.html")
    })
    s.Run()
}
toml
[database]
package utils
import (
    "github.com/gogf/gf/v2/database/gdb"
    "github.com/gogf/gf/v2/frame/g"
)
func InitDB() {
    err := g.DB().Init()
    if err != nil {
        g.Log().Fatalf("Failed to initialize database: %v", err)
    }
}
package utils
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
)
func GetEchartsData(r *ghttp.Request) {
    // 从数据库查询数据
    db := g.DB()
    records, err := db.Table("system_status_logs").All()
    if err != nil {
        r.Response.WriteJson(g.Map{
```

```
                "code": 500,
                "msg":    "Failed to query data from database",
            })
            return
        }
        var cpuData, memoryData, diskData []int
        var timeData []string
        for _, record := range records {
            timeData = append(timeData, record.GetString("monitor_time"))
            cpuData = append(cpuData, record.GetInt("cpu_usage"))
            memoryData = append(memoryData, record.GetInt("memory_usage"))
            diskData = append(diskData, record.GetInt("disk_usage"))
        }
        r.Response.WriteJson(g.Map{
            "code": 200,
            "data": g.Map{
                "time":     timeData,
                "cpu":      cpuData,
                "memory": memoryData,
                "disk":     diskData,
            },
        })
}
package models
import (
    "github.com/gogf/gf/v2/database/gdb"
    "github.com/gogf/gf/v2/frame/g"
)
type SystemStatus struct {
    DeviceName       string `json:"device_name"`
    DeviceIP         string `json:"device_ip"`
    CPUUsage         int       `json:"cpu_usage"`
    MemoryUsage      int       `json:"memory_usage"`
    DiskUsage        int       `json:"disk_usage"`
    NetworkBandwidth int `json:"network_bandwidth"`
    Status           string `json:"status"`
}
func GetSystemStatus() ([]SystemStatus, error) {
    db := g.DB()
    records, err := db.Table("system_status").All()
    if err != nil {
        return nil, err
    }
    var statusList []SystemStatus
    for _, record := range records {
        status := SystemStatus{
            DeviceName:       record.GetString("device_name"),
            DeviceIP:         record.GetString("device_ip"),
            CPUUsage:         record.GetInt("cpu_usage"),
            MemoryUsage:      record.GetInt("memory_usage"),
```

```
                DiskUsage:         record.GetInt("disk_usage"),
                NetworkBandwidth: record.GetInt("network_bandwidth"),
                Status:            record.GetString("status"),
            }
            statusList = append(statusList, status)
        }
        return statusList, nil
}
package handlers
import (
        "github.com/gogf/gf/v2/frame/g"
        "github.com/gogf/gf/v2/net/ghttp"
        "project/internal/models"
        "project/internal/utils"
)
func SystemStatusHandler(r *ghttp.Request) {
        // 获取系统状态数据
        statusList, err := models.GetSystemStatus()
        if err != nil {
            r.Response.WriteJson(g.Map{
                "code": 500,
                "msg":   "Failed to get system status",
            })
            return
        }
        r.Response.WriteJson(g.Map{
            "code": 200,
            "data": statusList,
        })
}
func EchartsDataHandler(r *ghttp.Request) {
        utils.GetEchartsData(r)
}
package models
import (
        "github.com/gogf/gf/v2/test/gtest"
        "testing"
)
func TestGetSystemStatus(t *testing.T) {
        gtest.C(t, func(t *gtest.T) {
            statusList, err := GetSystemStatus()
            t.AssertNil(err)
            t.AssertGreater(len(statusList), 0)
        })
}
package handlers
import (
        "github.com/gogf/gf/v2/net/ghttp"
        "github.com/gogf/gf/v2/test/gtest"
        "testing"
```

```
)
func TestSystemStatusHandler(t *testing.T) {
	s := ghttp.GetTestServer()
	s.BindHandler("/system-status", SystemStatusHandler)
	s.Start()
	defer s.Shutdown()
	gtest.C(t, func(t *gtest.T) {
		r, err := g.Client().Get(s.GetAddr() + "/system-status")
		t.AssertNil(err)
		defer r.Close()
		t.Assert(r.StatusCode, 200)
	})
}
```

```
html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://cdn.jsdelivr.net/npm/echarts@5.4.3/dist/echarts.min.js"></script>
    <link rel="stylesheet" href="/public/css/style.css">
</head>
<body>
    <div class="container">
        <div class="header">
            <h2><i>  </i>系统状态监控</h2>
            <div class="btn-group">
                <button class="btn btn-primary" id="refreshBtn"><i>  </i>刷新</button>
                <button class="btn btn-default" id="exportDataBtn"><i>  </i>导出数据</button>
                <button class="btn btn-primary" id="openModalBtn"><i>  </i>查看详情</button>
            </div>
        </div>
        <div class="form-container">
            <div class="form-item">
                <label>监控时间段</label>
                <div class="date-picker">
                    <input type="text" class="form-input" placeholder="开始日期" id="startDate">
                </div>
            </div>
            <div class="form-item">
                <div class="date-picker">
                    <input type="text" class="form-input" placeholder="结束日期" id="endDate">
                </div>
            </div>
            <div class="form-item">
                <label>监控设备</label>
                <select class="form-select">
                    <option value="all">所有设备</option>
                    <option value="device1">设备 1</option>
                    <option value="device2">设备 2</option>
```

```
                    <option value="device3">设备 3</option>
                </select>
            </div>
            <div class="form-item">
                <label>监控指标</label>
                <select class="form-select">
                    <option value="cpu">CPU 使用率</option>
                    <option value="memory">内存使用率</option>
                    <option value="disk">磁盘使用率</option>
                    <option value="network">网络带宽</option>
                </select>
            </div>
            <div class="form-item">
                <label>数据更新频率</label>
                <select class="form-select">
                    <option value="1m">1 分钟</option>
                    <option value="5m">5 分钟</option>
                    <option value="15m">15 分钟</option>
                    <option value="30m">30 分钟</option>
                </select>
            </div>
            <div class="form-item">
                <button class="btn btn-primary" style="width: 100%; margin-top: 22px;"><i>  </i>查询
</button>
            </div>
            <div class="form-item">
                <label>是否开启实时监控</label>
                <div class="switch-container">
                    <label class="switch">
                        <input type="checkbox" id="realTimeSwitch">
                        <span class="slider"></span>
                    </label>
                    <span>开启</span>
                </div>
            </div>
            <div class="form-item">
                <label>是否接收告警通知</label>
                <div class="switch-container">
                    <label class="switch">
                        <input type="checkbox" id="alarmSwitch">
                        <span class="slider"></span>
                    </label>
                    <span>开启</span>
                </div>
            </div>
            <div class="form-item">
                <label>告警阈值设置</label>
                <input type="number" class="form-input" placeholder="请输入阈值"
id="alarmThreshold">
            </div>
```

```
        </div>
        <div class="grid-container">
            <div class="card">
                <div class="card-header">
                    <h3><i>  </i>设备状态</h3>
                    <span class="status-label online">在线</span>
                </div>
                <div class="card-body">
                    <p>设备名称: <span id="deviceName"></span></p>
                    <p>设备 IP: <span id="deviceIP"></span></p>
                    <div class="progress-container">
                        <p>CPU 使用率</p>
                        <div class="progress">
                            <div class="progress-bar" id="cpuUsageBar" style="width: 0%;">0%</div>
                        </div>
                    </div>
                    <div class="progress-container">
                        <p>内存使用率</p>
                        <div class="progress">
                            <div class="progress-bar" id="memoryUsageBar" style="width:
0%;">0%</div>
                        </div>
                    </div>
                    <div class="progress-container">
                        <p>磁盘使用率</p>
                        <div class="progress">
                            <div class="progress-bar" id="diskUsageBar" style="width: 0%;">0%</div>
                        </div>
                    </div>
                </div>
            </div>
            <div class="card">
                <div class="card-header">
                    <h3><i>  </i>网络状态</h3>
                    <span class="status-label online">正常</span>
                </div>
                <div class="card-body">
                    <p>网络带宽: <span id="networkBandwidth"></span></p>
                    <p>当前使用带宽: <span id="currentBandwidth"></span></p>
                    <div class="progress-container">
                        <p>带宽使用率</p>
                        <div class="progress">
                            <div class="progress-bar" id="bandwidthUsageBar" style="width:
0%;">0%</div>
                        </div>
                    </div>
                    <div class="info-box">
                        <p>网络延迟: <span id="networkLatency"></span></p>
                        <p>丢包率: <span id="packetLossRate"></span></p>
                    </div>
```

```
                </div>
            </div>
        </div>
        <div class="table-container">
            <table>
                <thead>
                    <tr>
                        <th>监控时间</th>
                        <th>设备名称</th>
                        <th>CPU 使用率</th>
                        <th>内存使用率</th>
                        <th>磁盘使用率</th>
                        <th>网络带宽</th>
                        <th>状态</th>
                        <th>操作</th>
                    </tr>
                </thead>
                <tbody id="statusTableBody">
                </tbody>
            </table>
        </div>
        <div class="multi-chart-container">
            <div class="chart-container" id="cpuUsageChart"></div>
            <div class="chart-container" id="memoryUsageChart"></div>
            <div class="chart-container" id="diskUsageChart"></div>
        </div>
        <div class="tips">
            <div class="tips-title">使用提示</div>
            <div class="tips-content">
                <p>1. 可通过刷新按钮实时获取最新的系统状态数据。</p>
                <p>2. 导出数据功能可将当前监控数据保存为文件。</p>
                <p>3. 开启实时监控和告警通知功能，可及时掌握系统异常情况。</p>
            </div>
        </div>
    </div>
    <div id="detailModal" class="modal">
        <div class="modal-content">
            <div class="modal-header">
                <div class="modal-title">系统状态详情</div>
                <button class="close-btn" id="closeModal"><i>✖</i></button>
            </div>
            <div class="form-container" style="grid-template-columns: 1fr;">
                <div class="form-item">
                    <label>设备名称</label>
                    <input type="text" class="form-input" id="detailDeviceName" readonly>
                </div>
                <div class="form-item">
                    <label>设备 IP</label>
                    <input type="text" class="form-input" id="detailDeviceIP" readonly>
                </div>
```

```
                    <div class="form-item">
                        <label>CPU 使用率</label>
                        <input type="text" class="form-input" id="detailCpuUsage" readonly>
                    </div>
                    <div class="form-item">
                        <label>内存使用率</label>
                        <input type="text" class="form-input" id="detailMemoryUsage" readonly>
                    </div>
                    <div class="form-item">
                        <label>磁盘使用率</label>
                        <input type="text" class="form-input" id="detailDiskUsage" readonly>
                    </div>
                    <div class="form-item">
                        <label>网络带宽</label>
                        <input type="text" class="form-input" id="detailNetworkBandwidth" readonly>
                    </div>
                    <div class="form-item">
                        <label>状态</label>
                        <input type="text" class="form-input" id="detailStatus" readonly>
                    </div>
                    <div class="form-item">
                        <label>详细日志</label>
                        <textarea class="form-input" rows="5" id="detailLog" readonly></textarea>
                    </div>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-default" id="cancelDetailBtn">关闭</button>
                    <button class="btn btn-primary" id="confirmDetailBtn">确认</button>
                </div>
            </div>
        </div>
        <script src="/public/js/script.js"></script>
</body>
</html>
script
document.addEventListener('DOMContentLoaded', function () {
    // 初始化日期选择器
    document.getElementById('startDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    document.getElementById('endDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    // 数据控制
    const detailModal = document.getElementById('detailModal');
    const openModalBtn = document.getElementById('openModalBtn');
    const closeModal = document.getElementById('closeModal');
    const cancelDetailBtn = document.getElementById('cancelDetailBtn');
    const confirmDetailBtn = document.getElementById('confirmDetailBtn');
    openModalBtn.addEventListener('click', function () {
```

```
        detailModal.style.display = 'flex';
    });
    closeModal.addEventListener('click', function () {
        detailModal.style.display = 'none';
    });
    cancelDetailBtn.addEventListener('click', function () {
        detailModal.style.display = 'none';
    });
    confirmDetailBtn.addEventListener('click', function () {
        const statusLabel = document.querySelector('.status-label.online');
        statusLabel.style.backgroundColor = '#faad14';
        statusLabel.textContent = '注意';
        detailModal.style.display = 'none';
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function (event) {
        if (event.target === detailModal) {
            detailModal.style.display = 'none';
        }
    });
    // 获取系统状态数据
    fetch('/system-status')
      .then(response => response.json())
      .then(data => {
            if (data.code === 200) {
                const status = data.data[0];
                document.getElementById('deviceName').textContent = status.device_name;
                document.getElementById('deviceIP').textContent = status.device_ip;
                document.getElementById('cpuUsageBar').style.width = status.cpu_usage + '%';
                document.getElementById('cpuUsageBar').textContent = status.cpu_usage + '%';
                document.getElementById('memoryUsageBar').style.width = status.memory_usage + '%';
                document.getElementById('memoryUsageBar').textContent = status.memory_usage + '%';
                document.getElementById('diskUsageBar').style.width = status.disk_usage + '%';
                document.getElementById('diskUsageBar').textContent = status.disk_usage + '%';
                document.getElementById('networkBandwidth').textContent = status.network_bandwidth
+ 'Mbps';
                document.getElementById('currentBandwidth').textContent = status.network_bandwidth
* 0.2 + 'Mbps';
                document.getElementById('bandwidthUsageBar').style.width = 20 + '%';
                document.getElementById('bandwidthUsageBar').textContent = 20 + '%';
                document.getElementById('networkLatency').textContent = '10ms';
                document.getElementById('packetLossRate').textContent = '0%';
                // 填充表格数据
                const tableBody = document.getElementById('statusTableBody');
                data.data.forEach(status => {
                    const row = document.createElement('tr');
                    row.innerHTML = `
                        <td>${status.monitor_time}</td>
                        <td>${status.device_name}</td>
                        <td>${status.cpu_usage}%</td>
```

```
                    <td>${status.memory_usage}%</td>
                    <td>${status.disk_usage}%</td>
                    <td>${status.network_bandwidth}Mbps</td>
                    <td><span class="status-label online">在线</span></td>
                    <td>
                        <button class="action-btn" style="background-color: #1890ff; color: white;">查看详情</button>
                    </td>
                `;
                tableBody.appendChild(row);
            });
        }
    });
// 初始化图表
initCharts();
});
function initCharts() {
// 获取图表数据
fetch('/echarts-data')
  .then(response => response.json())
  .then(data => {
      if (data.code === 200) {
          const timeData = data.data.time;
          const cpuData = data.data.cpu;
          const memoryData = data.data.memory;
          const diskData = data.data.disk;
          // CPU 使用率图表
          const cpuUsageChart = echarts.init(document.getElementById('cpuUsageChart'));
          const cpuOption = {
              title: {
                  text: 'CPU 使用率',
                  left: 'center',
                  textStyle: {
                      color: '#333'
                  }
              },
              tooltip: {
                  trigger: 'axis'
              },
              xAxis: {
                  type: 'category',
                  data: timeData
              },
              yAxis: {
                  type: 'value'
              },
              series: [
                  {
                      data: cpuData,
                      type: 'line',
```

```
                smooth: true,
                itemStyle: {
                    color: '#1890ff'
                }
            }
        ]
    };
    cpuUsageChart.setOption(cpuOption);
    // 内存使用率图表
    const memoryUsageChart = echarts.init(document.getElementById('memoryUsageChart'));
    const memoryOption = {
        title: {
            text: '内存使用率',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'axis'
        },
        xAxis: {
            type: 'category',
            data: timeData
        },
        yAxis: {
            type: 'value'
        },
        series: [
            {
                data: memoryData,
                type: 'line',
                smooth: true,
                itemStyle: {
                    color: '#52c41a'
                }
            }
        ]
    };
    memoryUsageChart.setOption(memoryOption);
    // 磁盘使用率图表
    const diskUsageChart = echarts.init(document.getElementById('diskUsageChart'));
    const diskOption = {
        title: {
            text: '磁盘使用率',
            left: 'center',
            textStyle: {
                color: '#333'
            }
```

```
                    },
                    tooltip: {
                        trigger: 'axis'
                    },
                    xAxis: {
                        type: 'category',
                        data: timeData
                    },
                    yAxis: {
                        type: 'value'
                    },
                    series: [
                        {
                            data: diskData,
                            type: 'line',
                            smooth: true,
                            itemStyle: {
                                color: '#faad14'
                            }
                        }
                    ]
                };
                diskUsageChart.setOption(diskOption);
                // 窗口大小变化时重新调整图表大小
                window.addEventListener('resize', function () {
                    cpuUsageChart.resize();
                    memoryUsageChart.resize();
                    diskUsageChart.resize();
                });
            }
        });
}
```

字段名	类型	描述
id	int	主键
device_name	varchar(255)	设备名称
device_ip	varchar(255)	设备 IP 地址
cpu_usage	int	CPU 使用率
memory_usage	int	内存使用率
disk_usage	int	磁盘使用率
network_bandwidth	int	网络带宽
status	varchar(255)	设备状态

字段名	类型	描述
id	int	主键
monitor_time	datetime	监控时间
device_name	varchar(255)	设备名称
cpu_usage	int	CPU 使用率
memory_usage	int	内存使用率
disk_usage	int	磁盘使用率

```
package main
import (
    "github.com/gogf/gf/frame/g"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    // 注册路由
    handlers.RegisterRoutes(s)
    s.Run()
}
toml
[database]
/* 这里直接复制原 HTML 中的样式 */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
}
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
```

```
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    margin-bottom: 20px;
```

```
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
    background-size: 16px;
    transition: all 0.3s;
}
.form-select:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
    display: flex;
    align-items: center;
}
.switch {
    position: relative;
    display: inline-block;
    width: 50px;
```

```
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
```

```
.table-container {
    margin-top: 20px;
}
table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}
th, td {
    padding: 12px 16px;
    text-align: left;
    border-bottom: 1px solid #eaeaea;
}
th {
    background-color: #fafafa;
    font-weight: 500;
    color: #333;
}
tr:hover {
    background-color: #f5f5f5;
}
.action-btn {
    padding: 4px 8px;
    font-size: 12px;
    margin-right: 5px;
}
.chart-container {
    height: 300px;
    margin-top: 20px;
}
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    z-index: 1000;
    justify-content: center;
    align-items: center;
}
.modal-content {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    width: 600px;
    max-width: 90%;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
```

```
.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 10px;
    border-bottom: 1px solid #eaeaea;
}
.modal-title {
    font-size: 18px;
    font-weight: 600;
}
.close-btn {
    background: none;
    border: none;
    font-size: 20px;
    cursor: pointer;
    color: #999;
}
.modal-footer {
    display: flex;
    justify-content: flex-end;
    gap: 10px;
    margin-top: 20px;
    padding-top: 15px;
    border-top: 1px solid #eaeaea;
}
.tips {
    margin-top: 20px;
    padding: 15px;
    background-color: #f0f9ff;
    border-left: 4px solid #1890ff;
    border-radius: 4px;
}
.tips-title {
    font-weight: 600;
    margin-bottom: 8px;
    color: #1890ff;
}
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
/* 新增样式 */
.audio-player {
```

```
        width: 100%;
        margin-top: 10px;
}
.rating {
        color: #ffc107;
}
.tag {
        display: inline-block;
        padding: 4px 8px;
        background-color: #e9ecef;
        border-radius: 4px;
        margin-right: 5px;
        margin-top: 5px;
}
script
// 初始化日期选择器
document.addEventListener('DOMContentLoaded', function() {
        // 模拟日期选择器
        document.getElementById('startDate').addEventListener('focus', function() {
                this.type = 'date';
        });
        document.getElementById('endDate').addEventListener('focus', function() {
                this.type = 'date';
        });
        document.getElementById('newRecordDate').addEventListener('focus', function() {
                this.type = 'date';
        });
        // 数据控制
        const recordModal = document.getElementById('recordModal');
        const confirmModal = document.getElementById('confirmModal');
        const addRecordBtn = document.getElementById('addRecordBtn');
        const closeRecordModalBtn = document.getElementById('closeRecordModalBtn');
        const closeConfirmModalBtn = document.getElementById('closeConfirmModalBtn');
        const cancelRecordBtn = document.getElementById('cancelRecordBtn');
        const confirmRecordBtn = document.getElementById('confirmRecordBtn');
        const cancelDeleteBtn = document.getElementById('cancelDeleteBtn');
        const confirmDeleteBtn = document.getElementById('confirmDeleteBtn');
        addRecordBtn.addEventListener('click', function() {
                recordModal.style.display = 'flex';
        });
        closeRecordModalBtn.addEventListener('click', function() {
                recordModal.style.display = 'none';
        });
        cancelRecordBtn.addEventListener('click', function() {
                recordModal.style.display = 'none';
        });
        confirmRecordBtn.addEventListener('click', function() {
                // 发送新增记录请求
                fetch('/addRecord', {
                        method: 'POST',
```

```
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                interactionTime: document.getElementById('newRecordDate').value,
                userId: document.querySelector('#recordModal .form-input:nth-child(2)').value,
                interactionType: document.querySelector('#recordModal .form-select:nth-child(3)').value,
                voiceContent: document.querySelector('#recordModal textarea:nth-child(4)').value,
                voiceClarity: document.querySelector('#recordModal .form-select:nth-child(6)').value,
                interactionResult:
document.querySelector('#recordModal .form-select:nth-child(7)').value,
                rating: document.querySelector('#recordModal .form-select:nth-child(8)').value,
                tags: document.querySelector('#recordModal .form-input:nth-child(9)').value,
                remarks: document.querySelector('#recordModal textarea:nth-child(10)').value
            })
        })
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                alert('记录添加成功！');
                recordModal.style.display = 'none';
                // 刷新表格
                location.reload();
            } else {
                alert('记录添加失败：' + data.message);
            }
        });
});
// 表格中的删除按钮点击事件
const deleteButtons = document.querySelectorAll('.action-btn[style*="background-color: #ff4d4f"]');
deleteButtons.forEach(btn => {
    btn.addEventListener('click', function() {
        const recordId = this.dataset.id;
        confirmModal.dataset.id = recordId;
        confirmModal.style.display = 'flex';
    });
});
closeConfirmModalBtn.addEventListener('click', function() {
    confirmModal.style.display = 'none';
});
cancelDeleteBtn.addEventListener('click', function() {
    confirmModal.style.display = 'none';
});
confirmDeleteBtn.addEventListener('click', function() {
    const recordId = confirmModal.dataset.id;
    // 发送删除记录请求
    fetch('/deleteRecord/' + recordId, {
        method: 'DELETE'
    })
    .then(response => response.json())
```

```
            .then(data => {
                if (data.success) {
                    alert('记录删除成功！');
                    confirmModal.style.display = 'none';
                    // 刷新表格
                    location.reload();
                } else {
                    alert('记录删除失败：' + data.message);
                }
            });
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function(event) {
        if (event.target === recordModal) {
            recordModal.style.display = 'none';
        }
        if (event.target === confirmModal) {
            confirmModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
});
function initCharts() {
    // 交互类型统计图
    const interactionTypeChart = echarts.init(document.getElementById('interactionTypeChart'));
    // 从后端获取数据
    fetch('/getInteractionTypeData')
        .then(response => response.json())
        .then(data => {
            const interactionTypeOption = {
                title: {
                    text: '交互类型分布',
                    left: 'center',
                    textStyle: {
                        color: '#333'
                    }
                },
                tooltip: {
                    trigger: 'item',
                    formatter: '{a} <br/>{b}: {c} ({d}%)'
                },
                legend: {
                    orient: 'vertical',
                    right: 10,
                    top: 'center',
                    data: ['查询', '指令', '反馈']
                },
                series: [
                    {
```

```
                name: '交互类型',
                type: 'pie',
                radius: ['50%', '70%'],
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: data
            }
        ]
    };
    interactionTypeChart.setOption(interactionTypeOption);
});
// 交互成功率统计图
const successRateChart = echarts.init(document.getElementById('successRateChart'));
// 从后端获取数据
fetch('/getSuccessRateData')
    .then(response => response.json())
    .then(data => {
        const successRateOption = {
            title: {
                text: '交互成功率统计',
                left: 'center',
                textStyle: {
                    color: '#333'
                }
            },
            tooltip: {
                trigger: 'axis',
                axisPointer: {
                    type: 'shadow'
                }
            },
            grid: {
```

```
                left: '3%',
                right: '4%',
                bottom: '3%',
                containLabel: true
            },
            xAxis: {
                type: 'category',
                data: ['查询', '指令', '反馈'],
                axisLine: {
                    lineStyle: {
                        color: '#999'
                    }
                }
            },
            yAxis: {
                type: 'value',
                axisLine: {
                    lineStyle: {
                        color: '#999'
                    }
                }
            },
            series: [
                {
                    name: '成功率',
                    type: 'bar',
                    data: data,
                    barWidth: '40%',
                    label: {
                        show: true,
                        position: 'top'
                    }
                }
            ]
        };
        successRateChart.setOption(successRateOption);
    });
    // 窗口大小变化时重新调整图表大小
    window.addEventListener('resize', function() {
        interactionTypeChart.resize();
        successRateChart.resize();
    });
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
    "project/internal/utils"
)
```

```
func RegisterRoutes(s *ghttp.Server) {
    s.BindHandler("/", IndexHandler)
    s.BindHandler("/addRecord", AddRecordHandler)
    s.BindHandler("/deleteRecord/:id", DeleteRecordHandler)
    s.BindHandler("/getInteractionTypeData", GetInteractionTypeDataHandler)
    s.BindHandler("/getSuccessRateData", GetSuccessRateDataHandler)
}
func IndexHandler(r *ghttp.Request) {
    records, err := models.GetVoiceInteractionRecords()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    interactionTypeData, err := models.GetInteractionTypeData()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    successRateData, err := models.GetSuccessRateData()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    r.Response.WriteTpl("index.html", g.Map{
        "records":             records,
        "interactionTypeData": utils.ConvertToEchartsData(interactionTypeData),
        "successRateData":     utils.ConvertToEchartsData(successRateData),
    })
}
func AddRecordHandler(r *ghttp.Request) {
    var data struct {
        InteractionTime   string `json:"interactionTime"`
        UserId            string `json:"userId"`
        InteractionType   string `json:"interactionType"`
        VoiceContent      string `json:"voiceContent"`
        VoiceClarity      string `json:"voiceClarity"`
        InteractionResult string `json:"interactionResult"`
        Rating            string `json:"rating"`
        Tags              string `json:"tags"`
        Remarks           string `json:"remarks"`
    }
    if err := r.Parse(&data); err != nil {
        r.Response.WriteJson(g.Map{
            "success": false,
            "message": err.Error(),
        })
        return
    }
    err := models.AddVoiceInteractionRecord(data)
    if err != nil {
```

```
            r.Response.WriteJson(g.Map{
                "success": false,
                "message": err.Error(),
            })
            return
        }
        r.Response.WriteJson(g.Map{
            "success": true,
            "message": "记录添加成功",
        })
}
func DeleteRecordHandler(r *ghttp.Request) {
        recordId := r.GetString("id")
        err := models.DeleteVoiceInteractionRecord(recordId)
        if err != nil {
            r.Response.WriteJson(g.Map{
                "success": false,
                "message": err.Error(),
            })
            return
        }
        r.Response.WriteJson(g.Map{
            "success": true,
            "message": "记录删除成功",
        })
}
func GetInteractionTypeDataHandler(r *ghttp.Request) {
        data, err := models.GetInteractionTypeData()
        if err != nil {
            r.Response.WriteStatus(500)
            return
        }
        r.Response.WriteJson(utils.ConvertToEchartsData(data))
}
func GetSuccessRateDataHandler(r *ghttp.Request) {
        data, err := models.GetSuccessRateData()
        if err != nil {
            r.Response.WriteStatus(500)
            return
        }
        r.Response.WriteJson(utils.ConvertToEchartsData(data))
}
package models
import (
        "github.com/gogf/gf/database/gdb"
        "github.com/gogf/gf/frame/g"
)
type VoiceInteractionRecord struct {
        ID                  string `json:"id"`
        InteractionTime     string `json:"interactionTime"`
```

```
    UserId                string `json:"userId"`
    InteractionType       string `json:"interactionType"`
    VoiceContent          string `json:"voiceContent"`
    VoiceClarity          string `json:"voiceClarity"`
    InteractionResult string `json:"interactionResult"`
    Rating                string `json:"rating"`
    Tags                  string `json:"tags"`
    Remarks               string `json:"remarks"`
}
func GetVoiceInteractionRecords() ([]VoiceInteractionRecord, error) {
    var records []VoiceInteractionRecord
    err := g.DB().Table("voice_interaction_records").Scan(&records)
    return records, err
}
func AddVoiceInteractionRecord(data struct {
    InteractionTime   string `json:"interactionTime"`
    UserId                string `json:"userId"`
    InteractionType   string `json:"interactionType"`
    VoiceContent          string `json:"voiceContent"`
    VoiceClarity          string `json:"voiceClarity"`
    InteractionResult string `json:"interactionResult"`
    Rating                string `json:"rating"`
    Tags                  string `json:"tags"`
    Remarks               string `json:"remarks"`
}) error {
    _, err := g.DB().Table("voice_interaction_records").Data(g.Map{
        "interaction_time":     data.InteractionTime,
        "user_id":              data.UserId,
        "interaction_type":     data.InteractionType,
        "voice_content":        data.VoiceContent,
        "voice_clarity":        data.VoiceClarity,
        "interaction_result": data.InteractionResult,
        "rating":               data.Rating,
        "tags":                 data.Tags,
        "remarks":              data.Remarks,
    }).Insert()
    return err
}
func DeleteVoiceInteractionRecord(id string) error {
    _, err := g.DB().Table("voice_interaction_records").Where("id", id).Delete()
    return err
}
func GetInteractionTypeData() (gdb.Result, error) {
    return g.DB().Table("voice_interaction_records").Group("interaction_type").Fields("interaction_type,
COUNT(*) as count").Select()
}
func GetSuccessRateData() (gdb.Result, error) {
    return g.DB().Table("voice_interaction_records").Group("interaction_type").Fields("interaction_type,
SUM(CASE WHEN interaction_result = '成功' THEN 1 ELSE 0 END) / COUNT(*) * 100 as success_rate").Select()
}
```

```
package utils
import (
	"github.com/gogf/gf/frame/g"
	"github.com/gogf/gf/os/gcfg"
)
func InitDB() error {
	cfg := gcfg.New()
	err := cfg.SetFileName("config/config.toml")
	if err != nil {
		return err
	}
	err = g.DB().SetConfigNodeDefault(cfg.GetMap("database"))
	return err
}
package utils
import (
	"github.com/gogf/gf/database/gdb"
	"github.com/gogf/gf/frame/g"
)
func ConvertToEchartsData(data gdb.Result) []g.Map {
	var result []g.Map
	for _, row := range data {
		result = append(result, g.Map{
			"value": row["count"],
			"name":    row["interaction_type"],
		})
	}
	return result
}
html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
	<meta charset="UTF-8">
	<meta name="viewport" content="width=device-width, initial-scale=1.0">
	<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
	<script src="https://cdn.jsdelivr.net/npm/echarts@5.4.3/dist/echarts.min.js"></script>
	<link rel="stylesheet" href="/public/css/style.css">
</head>
<body>
	<div class="container">
		<div class="header">
			<h2><i>    </i>语音交互记录</h2>
			<div class="btn-group">
				<button class="btn btn-primary" id="addRecordBtn"><i>+</i>新增记录</button>
				<button class="btn btn-default" id="importBtn"><i>    </i>导入</button>
				<button class="btn btn-default" id="exportBtn"><i>    </i>导出</button>
				<button class="btn btn-primary" id="searchBtn"><i>    </i>搜索</button>
			</div>
		</div>
```

```
<div class="form-container">
    <div class="form-item">
        <label>交互时间</label>
        <div class="date-picker">
            <input type="text" class="form-input" placeholder="选择日期" id="startDate">
        </div>
    </div>
    <div class="form-item">
        <label>至</label>
        <div class="date-picker">
            <input type="text" class="form-input" placeholder="选择日期" id="endDate">
        </div>
    </div>
    <div class="form-item">
        <label>用户 ID</label>
        <input type="text" class="form-input" placeholder="请输入用户 ID">
    </div>
    <div class="form-item">
        <label>交互类型</label>
        <select class="form-select">
            <option value="">全部</option>
            <option value="查询">查询</option>
            <option value="指令">指令</option>
            <option value="反馈">反馈</option>
        </select>
    </div>
    <div class="form-item">
        <label>语音清晰度</label>
        <select class="form-select">
            <option value="">全部</option>
            <option value="高">高</option>
            <option value="中">中</option>
            <option value="低">低</option>
        </select>
    </div>
    <div class="form-item">
        <label>交互结果</label>
        <select class="form-select">
            <option value="">全部</option>
            <option value="成功">成功</option>
            <option value="失败">失败</option>
            <option value="部分成功">部分成功</option>
        </select>
    </div>
    <div class="form-item">
        <label>是否加急</label>
        <div class="switch-container">
            <label class="switch">
                <input type="checkbox">
                <span class="slider"></span>
```

```
                    </label>
                    <span>是</span>
                </div>
            </div>
            <div class="form-item">
                <label>是否有后续跟进</label>
                <div class="switch-container">
                    <label class="switch">
                        <input type="checkbox">
                        <span class="slider"></span>
                    </label>
                    <span>是</span>
                </div>
            </div>
            <div class="form-item">
                <label>关键词</label>
                <input type="text" class="form-input" placeholder="请输入关键词">
            </div>
            <div class="form-item">
                <label>评分范围</label>
                <input type="text" class="form-input" placeholder="如：3-5">
            </div>
            <div class="form-item">
                <label>标签</label>
                <input type="text" class="form-input" placeholder="多个标签用逗号分隔">
            </div>
            <div class="form-item">
                <button class="btn btn-primary" style="width: 100%; margin-top: 22px;"><i>  </i>高级
搜索</button>
            </div>
        </div>
        <div class="table-container">
            <table>
                <thead>
                    <tr>
                        <th>序号</th>
                        <th>交互时间</th>
                        <th>用户 ID</th>
                        <th>交互类型</th>
                        <th>语音内容</th>
                        <th>语音清晰度</th>
                        <th>交互结果</th>
                        <th>评分</th>
                        <th>标签</th>
                        <th>操作</th>
                    </tr>
                </thead>
                <tbody>
                    {{range $index, $record := .records}}
                    <tr>
```

```
                        <td>{{$index + 1}}</td>
                        <td>{{$record.InteractionTime}}</td>
                        <td>{{$record.UserId}}</td>
                        <td>{{$record.InteractionType}}</td>
                        <td>{{$record.VoiceContent}}<audio class="audio-player" controls
src="https://example.com/audio1.mp3"></audio></td>
                        <td>{{$record.VoiceClarity}}</td>
                        <td>{{$record.InteractionResult}}</td>
                        <td><span class="rating">{{strings.Repeat "★" $record.Rating}}</span></td>
                        <td>
                            {{range $tag := strings.Split $record.Tags ","}}
                            <span class="tag">{{$tag}}</span>
                            {{end}}
                        </td>
                        <td>
                            <button class="action-btn" style="background-color: #1890ff; color:
white;">查看详情</button>
                            <button class="action-btn" style="background-color: #ff4d4f; color:
white;" data-id="{{$record.ID}}">删除</button>
                        </td>
                    </tr>
                    {{end}}
                </tbody>
            </table>
        </div>
        <div class="tips">
            <div class="tips-title">使用提示</div>
            <div class="tips-content">
                <p>1. 语音交互记录可帮助您了解用户与智能工具的交互情况，请妥善管理。</p>
                <p>2. 可通过搜索功能快速定位所需记录。</p>
                <p>3. 删除记录操作不可恢复，请谨慎操作。</p>
            </div>
        </div>
    </div>
    <div class="container">
        <div class="header">
            <h2><i>  </i>交互统计</h2>
        </div>
        <div class="grid-container">
            <div class="chart-container" id="interactionTypeChart"></div>
            <div class="chart-container" id="successRateChart"></div>
        </div>
    </div>
    <div id="recordModal" class="modal">
        <div class="modal-content">
            <div class="modal-header">
                <div class="modal-title">新增语音交互记录</div>
                <button class="close-btn" id="closeRecordModalBtn">&times;</button>
            </div>
            <div class="form-container" style="grid-template-columns: 1fr;">
```

```
                    <div class="form-item">
                        <label>交互时间 <span style="color: red;">*</span></label>
                        <div class="date-picker">
                            <input type="text" class="form-input" placeholder="选择日期"
id="newRecordDate">
                        </div>
                    </div>
                    <div class="form-item">
                        <label>用户 ID <span style="color: red;">*</span></label>
                        <input type="text" class="form-input" placeholder="请输入用户 ID">
                    </div>
                    <div class="form-item">
                        <label>交互类型 <span style="color: red;">*</span></label>
package main
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    // 注册路由
    s.BindHandler("/", handlers.IndexHandler)
    s.BindHandler("/remote_desktop", handlers.RemoteDesktopHandler)
    s.Run()
}
toml
[database]
    [database.default]
        type = "mysql"
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
```

```
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
}
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
```

```
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    margin-bottom: 20px;
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
```

```
        background-size: 16px;
        transition: all 0.3s;
}
.form-select:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
        display: flex;
        align-items: center;
}
.switch {
        position: relative;
        display: inline-block;
        width: 50px;
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
```

```
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
.table-container {
        margin-top: 20px;
}
table {
        width: 100%;
        border-collapse: collapse;
        font-size: 14px;
}
th,
td {
        padding: 12px 16px;
        text-align: left;
        border-bottom: 1px solid #eaeaea;
}
th {
        background-color: #fafafa;
        font-weight: 500;
        color: #333;
}
tr:hover {
        background-color: #f5f5f5;
}
.action-btn {
        padding: 4px 8px;
        font-size: 12px;
        margin-right: 5px;
        border-radius: 2px;
        cursor: pointer;
        border: none;
}
.chart-container {
        height: 300px;
        margin-top: 20px;
}
```

```
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    z-index: 1000;
    justify-content: center;
    align-items: center;
}
.modal-content {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    width: 600px;
    max-width: 90%;
    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 10px;
    border-bottom: 1px solid #eaeaea;
}
.modal-title {
    font-size: 18px;
    font-weight: 600;
}
.close-btn {
    background: none;
    border: none;
    font-size: 20px;
    cursor: pointer;
    color: #999;
}
.modal-footer {
    display: flex;
    justify-content: flex-end;
    gap: 10px;
    margin-top: 20px;
    padding-top: 15px;
    border-top: 1px solid #eaeaea;
}
.tips {
    margin-top: 20px;
    padding: 15px;
```

```
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
}
.tips-content {
        font-size: 14px;
        color: #555;
}
.grid-container {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 20px;
}
/* 新增样式 */
.monitor-container {
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 20px;
        margin-bottom: 20px;
}
.monitor-item {
        background-color: #f0f9ff;
        border-radius: 8px;
        padding: 20px;
        text-align: center;
}
.monitor-item h3 {
        font-size: 18px;
        margin-bottom: 10px;
}
.monitor-item p {
        font-size: 24px;
        font-weight: 600;
        color: #1890ff;
}
.progress-container {
        margin-top: 20px;
}
.progress-bar {
        background-color: #e9ecef;
        border-radius: 4px;
        height: 20px;
        position: relative;
}
.progress {
```

```
        background-color: #1890ff;
        border-radius: 4px;
        height: 100%;
        width: 0;
        transition: width 0.3s;
}
.progress-label {
        position: absolute;
        top: 0;
        left: 50%;
        transform: translateX(-50%);
        font-size: 12px;
        line-height: 20px;
        color: white;
}
.device-status {
        display: flex;
        align-items: center;
        gap: 10px;
}
.status-dot {
        width: 10px;
        height: 10px;
        border-radius: 50%;
}
.status-dot.online {
        background-color: #52c41a;
}
.status-dot.offline {
        background-color: #ff4d4f;
}
script
document.addEventListener('DOMContentLoaded', function () {
        // 初始化日期选择器
        const datePickers = document.querySelectorAll('.date-picker input');
        datePickers.forEach(datePicker => {
                datePicker.addEventListener('focus', function () {
                        this.type = 'date';
                });
        });
        // 数据控制
        const connectionModal = document.getElementById('connectionModal');
        const confirmDisconnectModal = document.getElementById('confirmDisconnectModal');
        const addConnectionBtn = document.getElementById('addConnectionBtn');
        const closeConnectionModalBtn = document.getElementById('closeConnectionModalBtn');
        const                                closeConfirmDisconnectModalBtn                                =
document.getElementById('closeConfirmDisconnectModalBtn');
        const cancelConnectionBtn = document.getElementById('cancelConnectionBtn');
        const confirmConnectionBtn = document.getElementById('confirmConnectionBtn');
        const cancelDisconnectBtn = document.getElementById('cancelDisconnectBtn');
```

```
    const confirmDisconnectBtn = document.getElementById('confirmDisconnectBtn');
    const disconnectAllBtn = document.getElementById('disconnectAllBtn');
    addConnectionBtn.addEventListener('click', function () {
        connectionModal.style.display = 'flex';
    });
    closeConnectionModalBtn.addEventListener('click', function () {
        connectionModal.style.display = 'none';
    });
    cancelConnectionBtn.addEventListener('click', function () {
        connectionModal.style.display = 'none';
    });
    confirmConnectionBtn.addEventListener('click', function () {
        alert('连接添加成功！');
        connectionModal.style.display = 'none';
    });
    disconnectAllBtn.addEventListener('click', function () {
        confirmDisconnectModal.style.display = 'flex';
    });
    closeConfirmDisconnectModalBtn.addEventListener('click', function () {
        confirmDisconnectModal.style.display = 'none';
    });
    cancelDisconnectBtn.addEventListener('click', function () {
        confirmDisconnectModal.style.display = 'none';
    });
    confirmDisconnectBtn.addEventListener('click', function () {
        alert('所有连接已断开！');
        confirmDisconnectModal.style.display = 'none';
        const progress = document.getElementById('progress');
        progress.style.width = '0%';
        const progressLabel = progress.querySelector('.progress-label');
        progressLabel.textContent = '0%';
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function (event) {
        if (event.target === connectionModal) {
            connectionModal.style.display = 'none';
        }
        if (event.target === confirmDisconnectModal) {
            confirmDisconnectModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
});
function initCharts() {
    // 连接状态统计图
    const connectionStatusChart = echarts.init(document.getElementById('connectionStatusChart'));
    const connectionStatusOption = {
        title: {
            text: '连接状态分布',
```

```
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item',
            formatter: '{a} <br/>{b}: {c} ({d}%)'
        },
        legend: {
            orient: 'vertical',
            right: 10,
            top: 'center',
            data: ['在线', '离线']
        },
        series: [
            {
                name: '连接状态',
                type: 'pie',
                radius: ['50%', '70%'],
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: [
                    { value: 20, name: '在线', itemStyle: { color: '#52c41a' } },
                    { value: 5, name: '离线', itemStyle: { color: '#ff4d4f' } }
                ]
            }
        ]
    };
    connectionStatusChart.setOption(connectionStatusOption);
    // 连接类型统计图
    const connectionTypeChart = echarts.init(document.getElementById('connectionTypeChart'));
```

```
const connectionTypeOption = {
    title: {
        text: '连接类型分布',
        left: 'center',
        textStyle: {
            color: '#333'
        }
    },
    tooltip: {
        trigger: 'axis',
        axisPointer: {
            type: 'shadow'
        }
    },
    grid: {
        left: '3%',
        right: '4%',
        bottom: '3%',
        containLabel: true
    },
    xAxis: {
        type: 'value',
        axisLine: {
            lineStyle: {
                color: '#999'
            }
        }
    },
    yAxis: {
        type: 'category',
        data: ['RDP', 'VNC', 'SSH'],
        axisLine: {
            lineStyle: {
                color: '#999'
            }
        }
    },
    series: [
        {
            name: '数量',
            type: 'bar',
            data: [
                {
                    value: 15,
                    itemStyle: {
                        color: '#1890ff'
                    }
                },
                {
                    value: 8,
```

```
                        itemStyle: {
                            color: '#13c2c2'
                        }
                    },
                    {
                        value: 2,
                        itemStyle: {
                            color: '#722ed1'
                        }
                    }
                ],
                barWidth: '40%',
                label: {
                    show: true,
                    position: 'right'
                }
            }
        ]
    };
    connectionTypeChart.setOption(connectionTypeOption);
    // 窗口大小变化时重新调整图表大小
    window.addEventListener('resize', function () {
        connectionStatusChart.resize();
        connectionTypeChart.resize();
    });
}
package handlers
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
)
func IndexHandler(r *ghttp.Request) {
    r.Response.WriteTpl("index.html")
}
package handlers
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
    "project/internal/models"
    "project/internal/utils"
)
func RemoteDesktopHandler(r *ghttp.Request) {
    // 获取远程桌面数据
    remoteDesktopData, err := models.GetRemoteDesktopData()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    // 生成图表数据
    connectionStatusOption := utils.GenerateConnectionStatusOption(remoteDesktopData)
```

```
	connectionTypeOption := utils.GenerateConnectionTypeOption(remoteDesktopData)
	// 渲染模板
	r.Response.WriteTpl("remote_desktop.html", g.Map{
		"RemoteDesktopData":		remoteDesktopData,
		"ConnectionStatusOption":	connectionStatusOption,
		"ConnectionTypeOption":		connectionTypeOption,
	})
}
package models
import (
	"github.com/gogf/gf/v2/database/gdb"
	"github.com/gogf/gf/v2/frame/g"
)
// RemoteDesktopRecord 定义远程桌面记录结构体
type RemoteDesktopRecord struct {
	DeviceName	string `json:"device_name"`
	IPAddress	string `json:"ip_address"`
	ConnectionType string `json:"connection_type"`
	Status		string `json:"status"`
	ConnectionTime string `json:"connection_time"`
}
// GetRemoteDesktopData 从数据库获取远程桌面数据
func GetRemoteDesktopData() ([]RemoteDesktopRecord, error) {
	var records []RemoteDesktopRecord
	err := g.DB().Model("remote_desktop_records").Scan(&records)
	return records, err
}
package utils
import (
	"github.com/gogf/gf/v2/database/gdb"
	"github.com/gogf/gf/v2/frame/g"
)
// 初始化数据库连接
func init() {
	err := g.DB().Init()
	if err != nil {
		g.Log().Fatalf("Failed to initialize database: %v", err)
	}
}
package utils
import (
	"github.com/gogf/gf/v2/frame/g"
	"project/internal/models"
)
// GenerateConnectionStatusOption 生成连接状态图表配置
func GenerateConnectionStatusOption(data []models.RemoteDesktopRecord) g.Map {
	onlineCount := 0
	offlineCount := 0
	for _, record := range data {
		if record.Status == "在线" {
```

```
                onlineCount++
        } else {
                offlineCount++
        }
}
return g.Map{
        "title": g.Map{
                "text":    "连接状态分布",
                "left":    "center",
                "textStyle": g.Map{
                        "color": "#333",
                },
        },
        "tooltip": g.Map{
                "trigger": "item",
                "formatter": "{a} <br/>{b}: {c} ({d}%)",
        },
        "legend": g.Map{
                "orient": "vertical",
                "right":    10,
                "top":       "center",
                "data":     []string{"在线", "离线"},
        },
        "series": []g.Map{
                {
                        "name": "连接状态",
                        "type": "pie",
                        "radius": []string{"50%", "70%"},
                        "avoidLabelOverlap": false,
                        "itemStyle": g.Map{
                                "borderRadius": 10,
                                "borderColor":    "#fff",
                                "borderWidth":    2,
                        },
                        "label": g.Map{
                                "show": false,
                                "position": "center",
                        },
                        "emphasis": g.Map{
                                "label": g.Map{
                                        "show": true,
                                        "fontSize": "18",
                                        "fontWeight": "bold",
                                },
                        },
                        "labelLine": g.Map{
                                "show": false,
                        },
                        "data": []g.Map{
                                {
```

```
                                "value": onlineCount,
                                "name":    "在线",
                                "itemStyle": g.Map{
                                        "color": "#52c41a",
                                },
                        },
                        {
                                "value": offlineCount,
                                "name":    "离线",
                                "itemStyle": g.Map{
                                        "color": "#ff4d4f",
                                },
                        },
                },
        },
    },
}
// GenerateConnectionTypeOption  生成连接类型图表配置
func GenerateConnectionTypeOption(data []models.RemoteDesktopRecord) g.Map {
    rdpCount := 0
    vncCount := 0
    sshCount := 0
    for _, record := range data {
        switch record.ConnectionType {
        case "RDP":
            rdpCount++
        case "VNC":
            vncCount++
        case "SSH":
            sshCount++
        }
    }
    return g.Map{
        "title": g.Map{
            "text":    "连接类型分布",
            "left":    "center",
            "textStyle": g.Map{
                "color": "#333",
            },
        },
        "tooltip": g.Map{
            "trigger": "axis",
            "axisPointer": g.Map{
                "type": "shadow",
            },
        },
        "grid": g.Map{
            "left":    "3%",
            "right": "4%",
```

```
        "bottom": "3%",
        "containLabel": true,
    },
    "xAxis": g.Map{
        "type": "value",
        "axisLine": g.Map{
            "lineStyle": g.Map{
                "color": "#999",
            },
        },
    },
    "yAxis": g.Map{
        "type": "category",
        "data": []string{"RDP", "VNC", "SSH"},
        "axisLine": g.Map{
            "lineStyle": g.Map{
                "color": "#999",
            },
        },
    },
    "series": []g.Map{
        {
            "name": "数量",
            "type": "bar",
            "data": []g.Map{
                {
                    "value": rdpCount,
                    "itemStyle": g.Map{
                        "color": "#1890ff",
                    },
                },
                {
                    "value": vncCount,
                    "itemStyle": g.Map{
                        "color": "#13c2c2",
                    },
                },
                {
                    "value": sshCount,
                    "itemStyle": g.Map{
                        "color": "#722ed1",
                    },
                },
            },
            "barWidth": "40%",
            "label": g.Map{
                "show": true,
                "position": "right",
            },
        },
```

```
            },
        }
}
package handlers_test
import (
        "net/http"
        "net/http/httptest"
        "testing"
        "github.com/gogf/gf/v2/frame/g"
        "github.com/gogf/gf/v2/net/ghttp"
        "project/internal/handlers"
)
func TestRemoteDesktopHandler(t *testing.T) {
        s := g.Server()
        s.BindHandler("/remote_desktop", handlers.RemoteDesktopHandler)
        s.SetPort(8199)
        s.Start()
        defer s.Shutdown()
        req, err := http.NewRequest("GET", "http://127.0.0.1:8199/remote_desktop", nil)
        if err != nil {
                t.Fatal(err)
        }
        resp := httptest.NewRecorder()
        s.ServeHTTP(resp, req)
        if resp.Code != http.StatusOK {
                t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
        }
}
package models_test
import (
        "testing"
        "github.com/gogf/gf/v2/frame/g"
        "project/internal/models"
)
func TestGetRemoteDesktopData(t *testing.T) {
        records, err := models.GetRemoteDesktopData()
        if err != nil {
                t.Fatalf("Failed to get remote desktop data: %v", err)
        }
        g.Dump(records)
}
在 MySQL 中创建 `remote_desktop_records` 表:
CREATE TABLE remote_desktop_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    device_name VARCHAR(255) NOT NULL,
    ip_address VARCHAR(45) NOT NULL,
    connection_type VARCHAR(10) NOT NULL,
    status VARCHAR(10) NOT NULL,
    connection_time DATETIME NOT NULL
);
```

```
package main
import (
	"github.com/gogf/gf/frame/g"
	"github.com/gogf/gf/net/ghttp"
	"project/internal/handlers"
)
func main() {
	s := g.Server()
	s.BindHandler("/", handlers.IndexHandler)
	s.BindHandler("/video-monitoring", handlers.VideoMonitoringHandler)
	s.Run()
}
toml
[database]
* {
	margin: 0;
	padding: 0;
	box-sizing: border-box;
	font-family: "Microsoft YaHei", sans-serif;
}
body {
	background-color: #f5f5f5;
	color: #333;
	width: 1710px;
	margin: 0 auto;
	padding: 20px;
	background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
	background-color: #fff;
	border-radius: 8px;
	box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
	padding: 20px;
	margin-bottom: 20px;
}
.header {
	display: flex;
	justify-content: space-between;
	align-items: center;
	margin-bottom: 20px;
	padding-bottom: 15px;
	border-bottom: 1px solid #eaeaea;
}
.header h2 {
	font-size: 22px;
	color: #333;
	font-weight: 600;
	display: flex;
	align-items: center;
}
```

```
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    margin-bottom: 20px;
```

```
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
    background-size: 16px;
    transition: all 0.3s;
}
.form-select:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
    display: flex;
    align-items: center;
}
.switch {
    position: relative;
    display: inline-block;
    width: 50px;
```

```
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
```

```
.table-container {
    margin-top: 20px;
}
table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}
th,
td {
    padding: 12px 16px;
    text-align: left;
    border-bottom: 1px solid #eaeaea;
}
th {
    background-color: #fafafa;
    font-weight: 500;
    color: #333;
}
tr:hover {
    background-color: #f5f5f5;
}
.action-btn {
    padding: 4px 8px;
    font-size: 12px;
    margin-right: 5px;
    border-radius: 2px;
    cursor: pointer;
    border: none;
}
.chart-container {
    height: 300px;
    margin-top: 20px;
}
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    z-index: 1000;
    justify-content: center;
    align-items: center;
}
.modal-content {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
```

```
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
        margin-top: 20px;
        padding-top: 15px;
        border-top: 1px solid #eaeaea;
}
.tips {
        margin-top: 20px;
        padding: 15px;
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
}
.tips-content {
        font-size: 14px;
        color: #555;
}
.grid-container {
        display: grid;
        grid-template-columns: 1fr 1fr;
```

```
        gap: 20px;
}
.video-container {
        width: 100%;
        height: 400px;
        background-color: #000;
        margin-top: 20px;
        position: relative;
}
.video-container video {
        width: 100%;
        height: 100%;
        object-fit: cover;
}
.video-control {
        position: absolute;
        bottom: 10px;
        left: 10px;
        display: flex;
        gap: 10px;
}
.video-control .btn {
        background-color: rgba(0, 0, 0, 0.5);
        color: white;
}
.video-control .btn:hover {
        background-color: rgba(0, 0, 0, 0.7);
}
script
document.addEventListener('DOMContentLoaded', function () {
        // 初始化日期选择器
        const dateInputs = document.querySelectorAll('.date-picker input');
        dateInputs.forEach(input => {
                input.addEventListener('focus', function () {
                        this.type = 'datetime-local';
                });
        });
        // 数据控制
        const videoModal = document.getElementById('videoModal');
        const confirmDeleteModal = document.getElementById('confirmDeleteModal');
        const addVideoBtn = document.getElementById('addVideoBtn');
        const closeVideoModalBtn = document.getElementById('closeVideoModalBtn');
        const closeConfirmDeleteModalBtn = document.getElementById('closeConfirmDeleteModalBtn');
        const cancelVideoBtn = document.getElementById('cancelVideoBtn');
        const confirmVideoBtn = document.getElementById('confirmVideoBtn');
        const cancelDeleteVideoBtn = document.getElementById('cancelDeleteVideoBtn');
        const confirmDeleteVideoBtn = document.getElementById('confirmDeleteVideoBtn');
        addVideoBtn.addEventListener('click', function () {
                videoModal.style.display = 'flex';
        });
```

```
    closeVideoModalBtn.addEventListener('click', function () {
        videoModal.style.display = 'none';
    });
    cancelVideoBtn.addEventListener('click', function () {
        videoModal.style.display = 'none';
    });
    confirmVideoBtn.addEventListener('click', function () {
        // 发送新增视频源请求
        const formData = new FormData();
        const videoName = document.querySelector('#videoModal .form-input[placeholder="请输入视频名称"]').value;
        const videoUrl = document.querySelector('#videoModal .form-input[placeholder="请输入视频播放地址"]').value;
        const area = document.querySelector('#videoModal .form-select[option-value^="A 区"]').value;
        const clarity = document.querySelector('#videoModal .form-select[option-value^="高清"]').value;
        const startTime = document.querySelector('#videoModal .date-picker input[placeholder="选择开始时间"]').value;
        const endTime = document.querySelector('#videoModal .date-picker input[placeholder="选择结束时间"]').value;
        const status = document.querySelector('#videoModal .form-select[option-value^="正常"]').value;
        const isRecord = document.querySelector('#videoModal .switch input').checked;
        formData.append('video_name', videoName);
        formData.append('video_url', videoUrl);
        formData.append('area', area);
        formData.append('clarity', clarity);
        formData.append('start_time', startTime);
        formData.append('end_time', endTime);
        formData.append('status', status);
        formData.append('is_record', isRecord);
        fetch('/add-video', {
            method: 'POST',
            body: formData
        })
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                alert('视频源添加成功！');
                videoModal.style.display = 'none';
                // 刷新表格数据
                location.reload();
            } else {
                alert('视频源添加失败：' + data.message);
            }
        })
        .catch(error => {
            alert('请求出错：' + error.message);
        });
    });
    const deleteButtons = document.querySelectorAll('.action-btn[style*="background-color: #ff4d4f"]');
    deleteButtons.forEach(btn => {
```

```
        btn.addEventListener('click', function () {
            confirmDeleteModal.style.display = 'flex';
            const videoId = this.dataset.videoId;
            confirmDeleteVideoBtn.dataset.videoId = videoId;
        });
    });
    closeConfirmDeleteModalBtn.addEventListener('click', function () {
        confirmDeleteModal.style.display = 'none';
    });
    cancelDeleteVideoBtn.addEventListener('click', function () {
        confirmDeleteModal.style.display = 'none';
    });
    confirmDeleteVideoBtn.addEventListener('click', function () {
        const videoId = this.dataset.videoId;
        fetch('/delete-video/' + videoId, {
            method: 'DELETE'
        })
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                alert('视频源删除成功！');
                confirmDeleteModal.style.display = 'none';
                // 刷新表格数据
                location.reload();
            } else {
                alert('视频源删除失败：' + data.message);
            }
        })
        .catch(error => {
            alert('请求出错：' + error.message);
        });
    });
    window.addEventListener('click', function (event) {
        if (event.target === videoModal) {
            videoModal.style.display = 'none';
        }
        if (event.target === confirmDeleteModal) {
            confirmDeleteModal.style.display = 'none';
        }
    });
    // 视频播放控制
    const playVideo = document.getElementById('playVideo');
    const playBtn = document.getElementById('playBtn');
    const pauseBtn = document.getElementById('pauseBtn');
    const fullscreenBtn = document.getElementById('fullscreenBtn');
    playBtn.addEventListener('click', function () {
        playVideo.play();
    });
    pauseBtn.addEventListener('click', function () {
        playVideo.pause();
```

```
    });
    fullscreenBtn.addEventListener('click', function () {
        if (playVideo.requestFullscreen) {
            playVideo.requestFullscreen();
        } else if (playVideo.webkitRequestFullscreen) {
            playVideo.webkitRequestFullscreen();
        } else if (playVideo.msRequestFullscreen) {
            playVideo.msRequestFullscreen();
        }
    });
    // 初始化图表
    initCharts();
});
function initCharts() {
    // 视频状态统计图
    const statusChart = echarts.init(document.getElementById('videoStatusChart'));
    const areaChart = echarts.init(document.getElementById('areaDistributionChart'));
    // 获取视频状态统计数据
    fetch('/video-status-statistics')
    .then(response => response.json())
    .then(data => {
        const statusOption = {
            title: {
                text: '视频状态分布',
                left: 'center',
                textStyle: {
                    color: '#333'
                }
            },
            tooltip: {
                trigger: 'item',
                formatter: '{a} <br/>{b}: {c} ({d}%)'
            },
            legend: {
                orient: 'vertical',
                right: 10,
                top: 'center',
                data: ['正常', '故障', '维护']
            },
            series: [
                {
                    name: '视频状态',
                    type: 'pie',
                    radius: ['50%', '70%'],
                    avoidLabelOverlap: false,
                    itemStyle: {
                        borderRadius: 10,
                        borderColor: '#fff',
                        borderWidth: 2
                    },
```

```
                    label: {
                        show: false,
                        position: 'center'
                    },
                    emphasis: {
                        label: {
                            show: true,
                            fontSize: '18',
                            fontWeight: 'bold'
                        }
                    },
                    labelLine: {
                        show: false
                    },
                    data: [
                        { value: data.normalCount, name: '正常', itemStyle: { color: '#52c41a' } },
                        { value: data.faultCount, name: '故障', itemStyle: { color: '#ff4d4f' } },
                        { value: data.maintenanceCount, name: '维护', itemStyle: { color: '#faad14' } }
                    ]
                }
            ]
        };
        statusChart.setOption(statusOption);
})
.catch(error => {
        alert('获取视频状态统计数据出错:  ' + error.message);
});
// 获取区域分布统计数据
fetch('/area-distribution-statistics')
.then(response => response.json())
.then(data => {
        const areaOption = {
            title: {
                text: '监控区域分布',
                left: 'center',
                textStyle: {
                    color: '#333'
                }
            },
            tooltip: {
                trigger: 'axis',
                axisPointer: {
                    type: 'shadow'
                }
            },
            grid: {
                left: '3%',
                right: '4%',
                bottom: '3%',
                containLabel: true
```

```
        },
        xAxis: {
            type: 'value',
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        yAxis: {
            type: 'category',
            data: ['A 区', 'B 区', 'C 区'],
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        series: [
            {
                name: '数量',
                type: 'bar',
                data: [
                    {
                        value: data.aAreaCount,
                        itemStyle: {
                            color: '#1890ff'
                        }
                    },
                    {
                        value: data.bAreaCount,
                        itemStyle: {
                            color: '#13c2c2'
                        }
                    },
                    {
                        value: data.cAreaCount,
                        itemStyle: {
                            color: '#722ed1'
                        }
                    }
                ],
                barWidth: '40%',
                label: {
                    show: true,
                    position: 'right'
                }
            }
        ]
    };
```

```
            areaChart.setOption(areaOption);
        })
        .catch(error => {
            alert('获取区域分布统计数据出错: ' + error.message);
        });
        // 窗口大小变化时重新调整图表大小
        window.addEventListener('resize', function () {
            statusChart.resize();
            areaChart.resize();
        });
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
)
func IndexHandler(r *ghttp.Request) {
    r.Response.WriteTpl("index.html")
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
)
func VideoMonitoringHandler(r *ghttp.Request) {
    // 获取视频监控数据
    videos, err := models.GetAllVideos()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "success": false,
            "message": err.Error(),
        })
        return
    }
    // 获取视频状态统计数据
    statusStatistics, err := models.GetVideoStatusStatistics()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "success": false,
            "message": err.Error(),
        })
        return
    }
    // 获取区域分布统计数据
    areaStatistics, err := models.GetAreaDistributionStatistics()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "success": false,
            "message": err.Error(),
```

```
            })
            return
        }
        r.Response.WriteTpl("video-monitoring.html", g.Map{
            "videos":           videos,
            "statusStatistics":  statusStatistics,
            "areaStatistics":    areaStatistics,
        })
}
func AddVideoHandler(r *ghttp.Request) {
        videoName := r.GetString("video_name")
        videoUrl := r.GetString("video_url")
        area := r.GetString("area")
        clarity := r.GetString("clarity")
        startTime := r.GetString("start_time")
        endTime := r.GetString("end_time")
        status := r.GetString("status")
        isRecord := r.GetBool("is_record")
        err := models.AddVideo(videoName, videoUrl, area, clarity, startTime, endTime, status, isRecord)
        if err != nil {
            r.Response.WriteJson(g.Map{
                "success": false,
                "message": err.Error(),
            })
            return
        }
        r.Response.WriteJson(g.Map{
            "success": true,
            "message": "视频源添加成功",
        })
}
func DeleteVideoHandler(r *ghttp.Request) {
        videoId := r.GetString("id")
        err := models.DeleteVideo(videoId)
        if err != nil {
            r.Response.WriteJson(g.Map{
                "success": false,
                "message": err.Error(),
            })
            return
        }
        r.Response.WriteJson(g.Map{
            "success": true,
            "message": "视频源删除成功",
        })
}
package models
import (
        "github.com/gogf/gf/database/gdb"
        "github.com/gogf/gf/frame/g"
```

```
)
// Video 视频结构体
type Video struct {
    ID        int    `json:"id"`
    Name      string `json:"name"`
    Url       string `json:"url"`
    Area      string `json:"area"`
    Clarity   string `json:"clarity"`
    StartTime string `json:"start_time"`
    EndTime   string `json:"end_time"`
    Status    string `json:"status"`
    IsRecord  bool   `json:"is_record"`
}
// GetAllVideos 获取所有视频数据
func GetAllVideos() ([]Video, error) {
    var videos []Video
    err := g.DB().Table("videos").Structs(&videos)
    return videos, err
}
// AddVideo 添加视频数据
func AddVideo(name, url, area, clarity, startTime, endTime, status string, isRecord bool) error {
    _, err := g.DB().Table("videos").Data(g.Map{
        "name":       name,
        "url":        url,
        "area":       area,
        "clarity":    clarity,
        "start_time": startTime,
        "end_time":   endTime,
        "status":     status,
        "is_record":  isRecord,
    }).Insert()
    return err
}
// DeleteVideo 删除视频数据
func DeleteVideo(id string) error {
    _, err := g.DB().Table("videos").Where("id", id).Delete()
    return err
}
// VideoStatusStatistics 视频状态统计结构体
type VideoStatusStatistics struct {
    NormalCount      int `json:"normalCount"`
    FaultCount       int `json:"faultCount"`
    MaintenanceCount int `json:"maintenanceCount"`
}
// GetVideoStatusStatistics 获取视频状态统计数据
func GetVideoStatusStatistics() (*VideoStatusStatistics, error) {
    var statistics VideoStatusStatistics
    err := g.DB().Table("videos").Fields("COUNT(CASE WHEN status = '正常' THEN 1 END) as normal_count,
COUNT(CASE WHEN status = '故障' THEN 1 END) as fault_count, COUNT(CASE WHEN status = '维护' THEN
1 END) as maintenance_count").Struct(&statistics)
```

```
    return &statistics, err
}
// AreaDistributionStatistics 区域分布统计结构体
type AreaDistributionStatistics struct {
    AAreaCount int `json:"aAreaCount"`
    BAreaCount int `json:"bAreaCount"`
    CAreaCount int `json:"cAreaCount"`
}
// GetAreaDistributionStatistics 获取区域分布统计数据
func GetAreaDistributionStatistics() (*AreaDistributionStatistics, error) {
    var statistics AreaDistributionStatistics
    err := g.DB().Table("videos").Fields("COUNT(CASE WHEN area = 'A 区' THEN 1 END) as a_area_count,
COUNT(CASE WHEN area = 'B 区' THEN 1 END) as b_area_count, COUNT(CASE WHEN area = 'C 区' THEN 1
END) as c_area_count").Struct(&statistics)
    return &statistics, err
}
package utils
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/os/gcfg"
)
func InitDB() error {
    cfg := gcfg.Instance()
    link, err := cfg.GetString("database.link")
    if err != nil {
        return err
    }
    return g.DB().SetConfig(gdb.ConfigGroup{
        "default": {
            gdb.ConfigNode{
                Link: link,
            },
        },
    })
}
package utils
import (
    "github.com/gogf/gf/frame/g"
)
// GenerateEchartsOption 生成 ECharts 配置
func GenerateEchartsOption(title, xAxisData, yAxisData string) string {
    option := g.Map{
        "title": g.Map{
            "text":   title,
            "left":   "center",
            "textStyle": g.Map{
                "color": "#333",
            },
        },
        "tooltip": g.Map{
```

```
            "trigger": "axis",
            "axisPointer": g.Map{
                "type": "shadow",
            },
        },
        "grid": g.Map{
            "left":          "3%",
            "right":         "4%",
            "bottom":        "3%",
            "containLabel": true,
        },
        "xAxis": g.Map{
            "type": "category",
            "data": xAxisData,
            "axisLine": g.Map{
                "lineStyle": g.Map{
                    "color": "#999",
                },
            },
        },
        "yAxis": g.Map{
            "type": "value",
            "axisLine": g.Map{
                "lineStyle": g.Map{
                    "color": "#999",
                },
            },
        },
        "series": []g.Map{
            {
                "name": "数量",
                "type": "bar",
                "data": yAxisData,
                "barWidth": "40%",
                "label": g.Map{
                    "show": true,
                    "position": "right",
                },
            },
        },
    }
    return g.JsonEncode(option)
}
package handlers_test
import (
    "net/http"
    "net/http/httptest"
    "testing"
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
```

```
    "project/internal/handlers"
)
func TestVideoMonitoringHandler(t *testing.T) {
    s := g.Server()
    s.BindHandler("/video-monitoring", handlers.VideoMonitoringHandler)
    s.SetPort(8199)
    s.Start()
    defer s.Shutdown()
    req, _ := http.NewRequest("GET", "http://127.0.0.1:8199/video-monitoring", nil)
    resp := httptest.NewRecorder()
    s.ServeHTTP(resp, req)
    if resp.Code != http.StatusOK {
        t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
    }
}
package models_test
import (
    "testing"
    "github.com/gogf/gf/frame/g"
    "project/internal/models"
)
func TestGetAllVideos(t *testing.T) {
    videos, err := models.GetAllVideos()
    if err != nil {
        t.Errorf("Failed to get all videos: %v", err)
    }
    g.Dump(videos)
}
创建 `videos` 表用于存储视频监控数据:
CREATE TABLE `videos` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(255) NOT NULL,
    `url` varchar(255) NOT NULL,
    `area` varchar(255) NOT NULL,
    `clarity` varchar(255) NOT NULL,
    `start_time` datetime NOT NULL,
    `end_time` datetime NOT NULL,
    `status` varchar(255) NOT NULL,
    `is_record` tinyint(1) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
toml
[database]
    driver = "mysql"
package utils
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
)
// InitDB 初始化数据库连接
```

```
func InitDB() error {
	err := g.DB().SetConfigWithMap(g.Map{
		"default": gdb.ConfigGroup{
			gdb.ConfigNode{
				Link: g.Cfg().GetString("database.link"),
			},
		},
	})
	return err
}
package models
import (
	"github.com/gogf/gf/frame/g"
	"github.com/gogf/gf/util/gconv"
)
// HardwareStatus 硬件状态结构体
type HardwareStatus struct {
	ID                int       `json:"id"`
	Name              string    `json:"name"`
	Type              string    `json:"type"`
	IP                string    `json:"ip"`
	Status            string    `json:"status"`
	Temperature       string    `json:"temperature"`
	CPUUsage          string    `json:"cpu_usage"`
	MemoryUsage       string    `json:"memory_usage"`
	NetworkBandwidth string `json:"network_bandwidth"`
	DetectionTime    string `json:"detection_time"`
}
// GetHardwareStatusList 获取硬件状态列表
func GetHardwareStatusList() ([]HardwareStatus, error) {
	var list []HardwareStatus
	err := g.DB().Table("hardware_status").Scan(&list)
	return list, err
}
// AddHardwareStatus 添加硬件状态
func AddHardwareStatus(data g.Map) (int64, error) {
	result, err := g.DB().Table("hardware_status").Data(data).Insert()
	if err != nil {
		return 0, err
	}
	return result.LastInsertId()
}
// DeleteHardwareStatus 删除硬件状态
func DeleteHardwareStatus(id int) (int64, error) {
	result, err := g.DB().Table("hardware_status").Where("id", id).Delete()
	if err != nil {
		return 0, err
	}
	return result.RowsAffected()
}
```

```
// GetHardwareStatusCountByStatus 根据状态获取硬件数量
func GetHardwareStatusCountByStatus(status string) (int, error) {
    count, err := g.DB().Table("hardware_status").Where("status", status).Count()
    return count, err
}
// GetHardwareStatusCountByType 根据类型获取硬件数量
func GetHardwareStatusCountByType(hardwareType string) (int, error) {
    count, err := g.DB().Table("hardware_status").Where("type", hardwareType).Count()
    return count, err
}
package utils
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/util/gconv"
)
// GetStatusDistributionOption 获取硬件状态分布统计图配置
func GetStatusDistributionOption() g.Map {
    onlineCount, _ := models.GetHardwareStatusCountByStatus("在线")
    offlineCount, _ := models.GetHardwareStatusCountByStatus("离线")
    warningCount, _ := models.GetHardwareStatusCountByStatus("警告")
    return g.Map{
        "title": g.Map{
            "text":    "硬件状态分布",
            "left":    "center",
            "textStyle": g.Map{
                "color": "#333",
            },
        },
        "tooltip": g.Map{
            "trigger": "item",
            "formatter": "{a} <br/>{b}: {c} ({d}%)",
        },
        "legend": g.Map{
            "orient": "vertical",
            "right":    10,
            "top":       "center",
            "data":     []string{"在线", "离线", "警告"},
        },
        "series": []g.Map{
            {
                "name": "硬件状态",
                "type": "pie",
                "radius": []string{"50%", "70%"},
                "avoidLabelOverlap": false,
                "itemStyle": g.Map{
                    "borderRadius": 10,
                    "borderColor":    "#fff",
                    "borderWidth":    2,
                },
                "label": g.Map{
```

```
                "show":         false,
                "position": "center",
            },
            "emphasis": g.Map{
                "label": g.Map{
                    "show":         true,
                    "fontSize": "18",
                    "fontWeight": "bold",
                },
            },
            "labelLine": g.Map{
                "show": false,
            },
            "data": []g.Map{
                {
                    "value": onlineCount,
                    "name":    "在线",
                    "itemStyle": g.Map{
                        "color": "#52c41a",
                    },
                },
                {
                    "value": offlineCount,
                    "name":    "离线",
                    "itemStyle": g.Map{
                        "color": "#ff4d4f",
                    },
                },
                {
                    "value": warningCount,
                    "name":    "警告",
                    "itemStyle": g.Map{
                        "color": "#faad14",
                    },
                },
            },
        },
    },
}
// GetHardwareTypeDistributionOption  获取硬件类型分布统计图配置
func GetHardwareTypeDistributionOption() g.Map {
    serverCount, _ := models.GetHardwareStatusCountByType("服务器")
    routerCount, _ := models.GetHardwareStatusCountByType("路由器")
    switchCount, _ := models.GetHardwareStatusCountByType("交换机")
    storageCount, _ := models.GetHardwareStatusCountByType("存储设备")
    return g.Map{
        "title": g.Map{
            "text":    "硬件类型分布",
            "left":    "center",
```

```
        "textStyle": g.Map{
            "color": "#333",
        },
    },
    "tooltip": g.Map{
        "trigger": "axis",
        "axisPointer": g.Map{
            "type": "shadow",
        },
    },
    "grid": g.Map{
        "left":   "3%",
        "right": "4%",
        "bottom": "3%",
        "containLabel": true,
    },
    "xAxis": g.Map{
        "type": "value",
        "axisLine": g.Map{
            "lineStyle": g.Map{
                "color": "#999",
            },
        },
    },
    "yAxis": g.Map{
        "type": "category",
        "data": []string{"服务器", "路由器", "交换机", "存储设备"},
        "axisLine": g.Map{
            "lineStyle": g.Map{
                "color": "#999",
            },
        },
    },
    "series": []g.Map{
        {
            "name": "数量",
            "type": "bar",
            "data": []g.Map{
                {
                    "value": serverCount,
                    "itemStyle": g.Map{
                        "color": "#1890ff",
                    },
                },
                {
                    "value": routerCount,
                    "itemStyle": g.Map{
                        "color": "#13c2c2",
                    },
                },
```

```
                        {
                            "value": switchCount,
                            "itemStyle": g.Map{
                                "color": "#722ed1",
                            },
                        },
                        {
                            "value": storageCount,
                            "itemStyle": g.Map{
                                "color": "#f5222d",
                            },
                        },
                    },
                    "barWidth": "40%",
                    "label": g.Map{
                        "show":     true,
                        "position": "right",
                    },
                },
            },
        }
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
    "project/internal/utils"
)
// HardwareStatusHandler 硬件状态处理
func HardwareStatusHandler(r *ghttp.Request) {
    list, err := models.GetHardwareStatusList()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "获取硬件状态列表失败",
        })
        return
    }
    statusOption := utils.GetStatusDistributionOption()
    typeOption := utils.GetHardwareTypeDistributionOption()
    r.Response.WriteJson(g.Map{
        "code": 200,
        "msg":  "成功",
        "data": g.Map{
            "list":       list,
            "statusOption":  statusOption,
            "typeOption":    typeOption,
        },
    })
```

```
}
// AddHardwareStatusHandler 添加硬件状态处理
func AddHardwareStatusHandler(r *ghttp.Request) {
    data := r.GetMap()
    _, err := models.AddHardwareStatus(data)
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "添加硬件状态失败",
        })
        return
    }
    r.Response.WriteJson(g.Map{
        "code": 200,
        "msg":  "添加成功",
    })
}
// DeleteHardwareStatusHandler 删除硬件状态处理
func DeleteHardwareStatusHandler(r *ghttp.Request) {
    id := r.GetInt("id")
    _, err := models.DeleteHardwareStatus(id)
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "删除硬件状态失败",
        })
        return
    }
    r.Response.WriteJson(g.Map{
        "code": 200,
        "msg":  "删除成功",
    })
}
package main
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/handlers"
    "project/internal/utils"
)
func main() {
    // 初始化数据库
    err := utils.InitDB()
    if err != nil {
        g.Log().Errorf("数据库初始化失败: %v", err)
        return
    }
    s := g.Server()
    s.BindHandler("/hardware/status", handlers.HardwareStatusHandler)
    s.BindHandler("/hardware/add", handlers.AddHardwareStatusHandler)
```

```
    s.BindHandler("/hardware/delete", handlers.DeleteHardwareStatusHandler)
    s.Run()
}
script
document.addEventListener('DOMContentLoaded', function () {
    // 初始化日期选择器
    document.getElementById('startDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    // 数据控制
    const addHardwareModal = document.getElementById('addHardwareModal');
    const openModalBtn = document.getElementById('openModalBtn');
    const closeModalBtn = document.getElementById('closeModalBtn');
    const cancelAddBtn = document.getElementById('cancelAddBtn');
    const confirmAddBtn = document.getElementById('confirmAddBtn');
    openModalBtn.addEventListener('click', function () {
        addHardwareModal.style.display = 'flex';
    });
    closeModalBtn.addEventListener('click', function () {
        addHardwareModal.style.display = 'none';
    });
    cancelAddBtn.addEventListener('click', function () {
        addHardwareModal.style.display = 'none';
    });
    confirmAddBtn.addEventListener('click', function () {
        const name = document.querySelector('#addHardwareModal input[name="name"]').value;
        const type = document.querySelector('#addHardwareModal select[name="type"]').value;
        const ip = document.querySelector('#addHardwareModal input[name="ip"]').value;
        const status = document.querySelector('#addHardwareModal select[name="status"]').value;
        const description = document.querySelector('#addHardwareModal
textarea[name="description"]').value;
        fetch('/hardware/add', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                name: name,
                type: type,
                ip: ip,
                status: status,
                description: description
            })
        })
            .then(response => response.json())
            .then(data => {
                if (data.code === 200) {
                    alert('硬件添加成功！');
                    addHardwareModal.style.display = 'none';
                    initPage();
```

```
                } else {
                    alert(data.msg);
                }
            });
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function (event) {
        if (event.target === addHardwareModal) {
            addHardwareModal.style.display = 'none';
        }
    });
    // 初始化页面
    initPage();
});
function initPage() {
    fetch('/hardware/status')
        .then(response => response.json())
        .then(data => {
            if (data.code === 200) {
                renderTable(data.data.list);
                initCharts(data.data.statusOption, data.data.typeOption);
            } else {
                alert(data.msg);
            }
        });
}
function renderTable(list) {
    const tableBody = document.querySelector('.table-container tbody');
    tableBody.innerHTML = '';
    list.forEach(item => {
        const tr = document.createElement('tr');
        tr.innerHTML = `
            <td>${item.name}</td>
            <td>${item.type}</td>
            <td>${item.ip}</td>
            <td><span class="status-indicator ${getStatusClass(item.status)}"></span>${item.status}</td>
            <td>${item.temperature}</td>
            <td>${item.cpu_usage}</td>
            <td>${item.memory_usage}</td>
            <td>${item.network_bandwidth}</td>
            <td>${item.detection_time}</td>
            <td>
                <button class="action-btn" style="background-color: #1890ff; color: white;">查看详情</button>
                <button class="action-btn" style="background-color: #ff4d4f; color: white;" onclick="deleteHardware(${item.id})">删除</button>
            </td>
        `;
        tableBody.appendChild(tr);
```

```
        });
}
function getStatusClass(status) {
        switch (status) {
                case '在线':
                        return 'online';
                case '离线':
                        return 'offline';
                case '警告':
                        return 'warning';
                default:
                        return '';
        }
}
function initCharts(statusOption, typeOption) {
        // 硬件状态分布统计图
        const statusChart = echarts.init(document.getElementById('statusDistributionChart'));
        statusChart.setOption(statusOption);
        // 硬件类型分布统计图
        const typeChart = echarts.init(document.getElementById('hardwareTypeDistributionChart'));
        typeChart.setOption(typeOption);
        // 窗口大小变化时重新调整图表大小
        window.addEventListener('resize', function () {
                statusChart.resize();
                typeChart.resize();
        });
}
function deleteHardware(id) {
        if (confirm('确定要删除该硬件信息吗？')) {
                fetch(`/hardware/delete?id=${id}`)
                        .then(response => response.json())
                        .then(data => {
                                if (data.code === 200) {
                                        alert('删除成功！');
                                        initPage();
                                } else {
                                        alert(data.msg);
                                }
                        });
        }
}
* {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
        font-family: "Microsoft YaHei", sans-serif;
}
body {
        background-color: #f5f5f5;
        color: #333;
```

```
        width: 1710px;
        margin: 0 auto;
        padding: 20px;
        background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
        background-color: #fff;
        border-radius: 8px;
        box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
        padding: 20px;
        margin-bottom: 20px;
}
.header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 15px;
        border-bottom: 1px solid #eaeaea;
}
.header h2 {
        font-size: 22px;
        color: #333;
        font-weight: 600;
        display: flex;
        align-items: center;
}
.header h2 i {
        margin-right: 10px;
        color: #1890ff;
        font-size: 24px;
}
.btn-group {
        display: flex;
        gap: 10px;
}
.btn {
        padding: 8px 16px;
        border-radius: 4px;
        cursor: pointer;
        font-size: 14px;
        border: none;
        transition: all 0.3s;
        display: flex;
        align-items: center;
        justify-content: center;
}
.btn-primary {
        background-color: #1890ff;
        color: white;
```

```
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    margin-bottom: 20px;
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
```

```
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
    background-size: 16px;
    transition: all 0.3s;
}
.form-select:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
    display: flex;
    align-items: center;
}
.switch {
    position: relative;
    display: inline-block;
    width: 50px;
    height: 24px;
    margin-right: 10px;
}
.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}
.slider {
    position: absolute;
    cursor: pointer;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: #ccc;
    transition: .4s;
    border-radius: 24px;
}
.slider:before {
    position: absolute;
    content: "";
    height: 16px;
```

```
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
.table-container {
        margin-top: 20px;
}
table {
        width: 100%;
        border-collapse: collapse;
        font-size: 14px;
}
th, td {
        padding: 12px 16px;
        text-align: left;
        border-bottom: 1px solid #eaeaea;
}
th {
        background-color: #fafafa;
        font-weight: 500;
        color: #333;
}
tr:hover {
        background-color: #f5f5f5;
}
.action-btn {
        padding: 4px 8px;
```

```
        font-size: 12px;
        margin-right: 5px;
        border-radius: 2px;
        cursor: pointer;
        border: none;
}
.chart-container {
        height: 300px;
        margin-top: 20px;
}
.modal {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        background-color: rgba(0, 0, 0, 0.5);
        z-index: 1000;
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
```

```
    display: flex;
    justify-content: flex-end;
    gap: 10px;
    margin-top: 20px;
    padding-top: 15px;
    border-top: 1px solid #eaeaea;
}
.tips {
    margin-top: 20px;
    padding: 15px;
    background-color: #f0f9ff;
    border-left: 4px solid #1890ff;
    border-radius: 4px;
}
.tips-title {
    font-weight: 600;
    margin-bottom: 8px;
    color: #1890ff;
}
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
/* 新增样式 */
.status-indicator {
    display: inline-block;
    width: 10px;
    height: 10px;
    border-radius: 50%;
    margin-right: 5px;
}
.status-indicator.online {
    background-color: #52c41a;
}
.status-indicator.offline {
    background-color: #ff4d4f;
}
.status-indicator.warning {
    background-color: #faad14;
}
```

创建 `hardware_status` 表：

```
CREATE TABLE `hardware_status` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(255) NOT NULL,
    `type` varchar(255) NOT NULL,
```

```
    `ip` varchar(255) NOT NULL,
    `status` varchar(255) NOT NULL,
    `temperature` varchar(255) NOT NULL,
    `cpu_usage` varchar(255) NOT NULL,
    `memory_usage` varchar(255) NOT NULL,
    `network_bandwidth` varchar(255) NOT NULL,
    `detection_time` datetime NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
package main
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    // 注册路由
    s.BindHandler("/", handlers.IndexHandler)
    s.BindHandler("/software-update", handlers.SoftwareUpdateHandler)
    s.Run()
}
toml
[database]
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
```

```
        padding-bottom: 15px;
        border-bottom: 1px solid #eaeaea;
}
.header h2 {
        font-size: 22px;
        color: #333;
        font-weight: 600;
        display: flex;
        align-items: center;
}
.header h2 i {
        margin-right: 10px;
        color: #1890ff;
        font-size: 24px;
}
.btn-group {
        display: flex;
        gap: 10px;
}
.btn {
        padding: 8px 16px;
        border-radius: 4px;
        cursor: pointer;
        font-size: 14px;
        border: none;
        transition: all 0.3s;
        display: flex;
        align-items: center;
        justify-content: center;
}
.btn-primary {
        background-color: #1890ff;
        color: white;
}
.btn-primary:hover {
        background-color: #40a9ff;
}
.btn-default {
        background-color: #f0f0f0;
        color: #333;
        border: 1px solid #d9d9d9;
}
.btn-default:hover {
        background-color: #e6e6e6;
}
.btn-danger {
        background-color: #ff4d4f;
        color: white;
}
.btn-danger:hover {
```

```
        background-color: #ff7875;
}
.btn i {
        margin-right: 5px;
}
.form-container {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        gap: 20px;
        margin-bottom: 20px;
}
.form-item {
        margin-bottom: 15px;
}
.form-item label {
        display: block;
        margin-bottom: 8px;
        font-size: 14px;
        color: #666;
}
.form-input {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
        border-radius: 4px;
        font-size: 14px;
        transition: all 0.3s;
}
.form-input:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
        border-radius: 4px;
        font-size: 14px;
        appearance: none;
        background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
        background-size: 16px;
        transition: all 0.3s;
}
.form-select:focus {
        border-color: #1890ff;
        outline: none;
```

```
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
        display: flex;
        align-items: center;
}
.switch {
        position: relative;
        display: inline-block;
        width: 50px;
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
```

```
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
.table-container {
        margin-top: 20px;
}
table {
        width: 100%;
        border-collapse: collapse;
        font-size: 14px;
}
th, td {
        padding: 12px 16px;
        text-align: left;
        border-bottom: 1px solid #eaeaea;
}
th {
        background-color: #fafafa;
        font-weight: 500;
        color: #333;
}
tr:hover {
        background-color: #f5f5f5;
}
.action-btn {
        padding: 4px 8px;
        font-size: 12px;
        margin-right: 5px;
        border-radius: 2px;
        cursor: pointer;
        border: none;
}
.chart-container {
        height: 300px;
        margin-top: 20px;
}
.modal {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
```

```
        background-color: rgba(0, 0, 0, 0.5);
        z-index: 1000;
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
        margin-top: 20px;
        padding-top: 15px;
        border-top: 1px solid #eaeaea;
}
.tips {
        margin-top: 20px;
        padding: 15px;
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
```

```
        color: #1890ff;
}
.tips-content {
        font-size: 14px;
        color: #555;
}
.grid-container {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 20px;
}
script
// 页面加载完成后执行的代码
document.addEventListener('DOMContentLoaded', function() {
        // 初始化日期选择器
        document.getElementById('releaseDatePicker').addEventListener('focus', function() {
                this.type = 'date';
        });
        // 数据控制
        const updateModal = document.getElementById('updateModal');
        const confirmDeleteModal = document.getElementById('confirmDeleteModal');
        const addUpdateBtn = document.getElementById('addUpdateBtn');
        const closeUpdateModalBtn = document.getElementById('closeUpdateModalBtn');
        const closeConfirmDeleteModalBtn = document.getElementById('closeConfirmDeleteModalBtn');
        const cancelUpdateBtn = document.getElementById('cancelUpdateBtn');
        const confirmUpdateBtn = document.getElementById('confirmUpdateBtn');
        const cancelDeleteUpdateBtn = document.getElementById('cancelDeleteUpdateBtn');
        const confirmDeleteUpdateBtn = document.getElementById('confirmDeleteUpdateBtn');
        // 点击新增更新按钮，显示新增更新数据
        addUpdateBtn.addEventListener('click', function() {
                updateModal.style.display = 'flex';
        });
        // 点击关闭数据按钮，隐藏新增更新数据
        closeUpdateModalBtn.addEventListener('click', function() {
                updateModal.style.display = 'none';
        });
        // 点击取消按钮，隐藏新增更新数据
        cancelUpdateBtn.addEventListener('click', function() {
                updateModal.style.display = 'none';
        });
        // 点击确认按钮，隐藏新增更新数据，并模拟更新添加成功提示
        confirmUpdateBtn.addEventListener('click', function() {
                alert('更新添加成功！');
                updateModal.style.display = 'none';
        });
        // 表格中的删除按钮点击事件，显示删除确认数据
        const deleteUpdateButtons = document.querySelectorAll('.action-btn[style*="background-color:
#ff4d4f"]');
        deleteUpdateButtons.forEach(btn => {
                btn.addEventListener('click', function() {
```

```
            confirmDeleteModal.style.display = 'flex';
        });
    });
    // 点击关闭删除确认数据按钮，隐藏删除确认数据
    closeConfirmDeleteModalBtn.addEventListener('click', function() {
        confirmDeleteModal.style.display = 'none';
    });
    // 点击取消删除按钮，隐藏删除确认数据
    cancelDeleteUpdateBtn.addEventListener('click', function() {
        confirmDeleteModal.style.display = 'none';
    });
    // 点击确认删除按钮，隐藏删除确认数据，并模拟更新删除成功提示
    confirmDeleteUpdateBtn.addEventListener('click', function() {
        alert('更新删除成功！');
        confirmDeleteModal.style.display = 'none';
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function(event) {
        if (event.target === updateModal) {
            updateModal.style.display = 'none';
        }
        if (event.target === confirmDeleteModal) {
            confirmDeleteModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
});
// 初始化图表的函数
function initCharts() {
    // 更新状态统计图
    const statusChart = echarts.init(document.getElementById('updateStatusChart'));
    const statusOption = {
        title: {
            text: '更新状态分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item',
            formatter: '{a} <br/>{b}: {c} ({d}%)'
        },
        legend: {
            orient: 'vertical',
            right: 10,
            top: 'center',
            data: ['待发布', '已发布', '已撤回']
        },
```

```
        series: [
            {
                name: '更新状态',
                type: 'pie',
                radius: ['50%', '70%'],
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: [
                    { value: 10, name: '待发布', itemStyle: { color: '#1890ff' } },
                    { value: 30, name: '已发布', itemStyle: { color: '#52c41a' } },
                    { value: 5, name: '已撤回', itemStyle: { color: '#ff4d4f' } }
                ]
            }
        ]
    };
    statusChart.setOption(statusOption);
    // 更新类型统计图
    const typeChart = echarts.init(document.getElementById('updateTypeChart'));
    const typeOption = {
        title: {
            text: '更新类型分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'axis',
            axisPointer: {
                type: 'shadow'
            }
        },
```

```
grid: {
    left: '3%',
    right: '4%',
    bottom: '3%',
    containLabel: true
},
xAxis: {
    type: 'value',
    axisLine: {
        lineStyle: {
            color: '#999'
        }
    }
},
yAxis: {
    type: 'category',
    data: ['功能更新', '安全更新', '性能优化'],
    axisLine: {
        lineStyle: {
            color: '#999'
        }
    }
},
series: [
    {
        name: '数量',
        type: 'bar',
        data: [
            {
                value: 20,
                itemStyle: {
                    color: '#1890ff'
                }
            },
            {
                value: 10,
                itemStyle: {
                    color: '#13c2c2'
                }
            },
            {
                value: 5,
                itemStyle: {
                    color: '#722ed1'
                }
            }
        ],
        barWidth: '40%',
        label: {
            show: true,
```

```
                        position: 'right'
                    }
                }
            ]
        };
        typeChart.setOption(typeOption);
        // 窗口大小变化时重新调整图表大小
        window.addEventListener('resize', function() {
            statusChart.resize();
            typeChart.resize();
        });
}
```

```
html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://cdn.jsdelivr.net/npm/echarts@5.4.3/dist/echarts.min.js"></script>
    <link rel="stylesheet" href="/public/css/style.css">
</head>
<body>
    <div class="container">
        <div class="header">
            <h2><i>  </i>软件更新管理</h2>
            <div class="btn-group">
                <button class="btn btn-primary" id="addUpdateBtn"><i>+</i>新增更新</button>
                <button class="btn btn-default" id="importUpdateBtn"><i>  </i>导入更新</button>
                <button class="btn btn-default" id="exportUpdateBtn"><i>  </i>导出更新</button>
            </div>
        </div>
        <div class="form-container">
            <div class="form-item">
                <label>更新名称</label>
                <input type="text" class="form-input" placeholder="请输入更新名称">
            </div>
            <div class="form-item">
                <label>更新版本号</label>
                <input type="text" class="form-input" placeholder="请输入更新版本号">
            </div>
            <div class="form-item">
                <label>更新类型</label>
                <select class="form-select">
                    <option value="">全部</option>
                    <option value="功能更新">功能更新</option>
                    <option value="安全更新">安全更新</option>
                    <option value="性能优化">性能优化</option>
                </select>
            </div>
            <div class="form-item">
```

```
                <label>更新大小</label>
                <input type="text" class="form-input" placeholder="请输入更新大小（MB）">
            </div>
            <div class="form-item">
                <label>更新状态</label>
                <select class="form-select">
                    <option value="待发布">待发布</option>
                    <option value="已发布">已发布</option>
                    <option value="已撤回">已撤回</option>
                </select>
            </div>
            <div class="form-item">
                <label>发布时间</label>
                <div class="date-picker">
                    <input type="text" class="form-input" placeholder="选择日期"
id="releaseDatePicker">
                </div>
            </div>
            <div class="form-item">
                <label>自动更新开关</label>
                <div class="switch-container">
                    <label class="switch">
                        <input type="checkbox" checked>
                        <span class="slider"></span>
                    </label>
                    <span>启用</span>
                </div>
            </div>
            <div class="form-item">
                <label>强制更新开关</label>
                <div class="switch-container">
                    <label class="switch">
                        <input type="checkbox">
                        <span class="slider"></span>
                    </label>
                    <span>启用</span>
                </div>
            </div>
            <div class="form-item">
                <button class="btn btn-primary" style="width: 100%; margin-top: 22px;"><i>  </i>搜索
</button>
            </div>
        </div>
        <div class="table-container">
            <table>
                <thead>
                    <tr>
                        <th>更新名称</th>
                        <th>更新版本号</th>
                        <th>更新类型</th>
```

```
                        <th>更新大小</th>
                        <th>更新状态</th>
                        <th>发布时间</th>
                        <th>自动更新</th>
                        <th>强制更新</th>
                        <th>操作</th>
                    </tr>
                </thead>
                <tbody>
                    {{range .Updates}}
                    <tr>
                        <td>{{.UpdateName}}</td>
                        <td>{{.VersionNumber}}</td>
                        <td>{{.UpdateType}}</td>
                        <td>{{.UpdateSize}}MB</td>
                        <td><span style="color: {{if eq .UpdateStatus "已发布"}}#52c41a{{else if eq .UpdateStatus "已撤回"}}#ff4d4f{{end}}">{{.UpdateStatus}}</span></td>
                        <td>{{.ReleaseTime}}</td>
                        <td><span style="color: {{if eq .AutoUpdate "启用"}}#52c41a{{else}}#ff4d4f{{end}}">{{.AutoUpdate}}</span></td>
                        <td><span style="color: {{if eq .ForceUpdate "启用"}}#52c41a{{else}}#ff4d4f{{end}}">{{.ForceUpdate}}</span></td>
                        <td>
                            <button class="action-btn" style="background-color: #1890ff; color: white;">编辑</button>
                            <button class="action-btn" style="background-color: #ff4d4f; color: white;">删除</button>
                        </td>
                    </tr>
                    {{end}}
                </tbody>
            </table>
        </div>
        <div class="tips">
            <div class="tips-title">使用提示</div>
            <div class="tips-content">
                <p>1. 软件更新管理用于发布和管理软件的更新信息，请谨慎操作。</p>
                <p>2. 已发布的更新可以撤回，但撤回后无法再次发布。</p>
                <p>3. 强制更新会在用户下次启动软件时自动下载并安装。</p>
            </div>
        </div>
    </div>
    <div class="container">
        <div class="header">
            <h2><i>  </i>更新统计</h2>
        </div>
        <div class="grid-container">
            <div class="chart-container" id="updateStatusChart"></div>
            <div class="chart-container" id="updateTypeChart"></div>
        </div>
```

```
</div>
<div id="updateModal" class="modal">
    <div class="modal-content">
        <div class="modal-header">
            <div class="modal-title">新增软件更新</div>
            <button class="close-btn" id="closeUpdateModalBtn">&times;</button>
        </div>
        <div class="form-container" style="grid-template-columns: 1fr;">
            <div class="form-item">
                <label>更新名称 <span style="color: red;">*</span></label>
                <input type="text" class="form-input" placeholder="请输入更新名称">
            </div>
            <div class="form-item">
                <label>更新版本号 <span style="color: red;">*</span></label>
                <input type="text" class="form-input" placeholder="请输入更新版本号">
            </div>
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
                <div class="form-item">
                    <label>更新类型 <span style="color: red;">*</span></label>
                    <select class="form-select">
                        <option value="功能更新">功能更新</option>
                        <option value="安全更新">安全更新</option>
                        <option value="性能优化">性能优化</option>
                    </select>
                </div>
                <div class="form-item">
                    <label>更新大小 <span style="color: red;">*</span></label>
                    <input type="text" class="form-input" placeholder="请输入更新大小（MB）">
                </div>
            </div>
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
                <div class="form-item">
                    <label>更新状态 <span style="color: red;">*</span></label>
                    <select class="form-select">
                        <option value="待发布">待发布</option>
                        <option value="已发布">已发布</option>
                    </select>
                </div>
                <div class="form-item">
                    <label>发布时间</label>
                    <div class="date-picker">
                        <input type="text" class="form-input" placeholder="选择日期">
                    </div>
                </div>
            </div>
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
                <div class="form-item">
                    <label>自动更新开关 <span style="color: red;">*</span></label>
                    <div class="switch-container">
                        <label class="switch">
```

```
                                <input type="checkbox" checked>
                                <span class="slider"></span>
                            </label>
                            <span>启用</span>
                        </div>
                    </div>
                    <div class="form-item">
                        <label>强制更新开关</label>
                        <div class="switch-container">
                            <label class="switch">
                                <input type="checkbox">
                                <span class="slider"></span>
                            </label>
                            <span>启用</span>
                        </div>
                    </div>
                </div>
                <div class="form-item">
                    <label>更新描述信息</label>
                    <textarea class="form-input" rows="3" placeholder="请输入更新描述信息
"></textarea>
                </div>
            </div>
            <div class="modal-footer">
                <button class="btn btn-default" id="cancelUpdateBtn">取消</button>
                <button class="btn btn-primary" id="confirmUpdateBtn">确认</button>
            </div>
        </div>
    </div>
    <div id="confirmDeleteModal" class="modal">
        <div class="modal-content" style="width: 400px;">
            <div class="modal-header">
                <div class="modal-title">操作确认</div>
                <button class="close-btn" id="closeConfirmDeleteModalBtn">&times;</button>
            </div>
            <div style="padding: 20px 0; text-align: center;">
                <p>确定要删除这条软件更新吗？此操作不可恢复。</p>
            </div>
            <div class="modal-footer">
                <button class="btn btn-default" id="cancelDeleteUpdateBtn">取消</button>
                <button class="btn btn-danger" id="confirmDeleteUpdateBtn">确认删除</button>
            </div>
        </div>
    </div>
    <script src="/public/js/script.js"></script>
</body>
</html>
package handlers
import (
    "github.com/gogf/gf/v2/frame/g"
```

```
	"github.com/gogf/gf/v2/net/ghttp"
	"project/internal/models"
)
func SoftwareUpdateHandler(r *ghttp.Request) {
	// 从数据库获取软件更新数据
	updates, err := models.GetSoftwareUpdates()
	if err != nil {
		r.Response.WriteStatus(500)
		return
	}
	// 渲染模板
	r.Response.WriteTpl("software_update.html", g.Map{
		"Updates": updates,
	})
}
package models
import (
	"github.com/gogf/gf/v2/database/gdb"
	"github.com/gogf/gf/v2/frame/g"
)
// SoftwareUpdate 软件更新结构体
type SoftwareUpdate struct {
	UpdateName    string `json:"update_name"`
	VersionNumber string `json:"version_number"`
	UpdateType    string `json:"update_type"`
	UpdateSize    int    `json:"update_size"`
	UpdateStatus  string `json:"update_status"`
	ReleaseTime   string `json:"release_time"`
	AutoUpdate    string `json:"auto_update"`
	ForceUpdate   string `json:"force_update"`
}
// GetSoftwareUpdates 从数据库获取软件更新数据
func GetSoftwareUpdates() ([]SoftwareUpdate, error) {
	var updates []SoftwareUpdate
	err := g.DB().Model("software_updates").Scan(&updates)
	return updates, err
}
package utils
import (
	"github.com/gogf/gf/v2/database/gdb"
	"github.com/gogf/gf/v2/frame/g"
)
// InitDB 初始化数据库连接
func InitDB() error {
	return g.DB().Init()
}
package utils
import (
	"github.com/gogf/gf/v2/frame/g"
)
```

```
// GenerateEchartsOption 生成 Echarts 配置
func GenerateEchartsOption() g.Map {
	return g.Map{}
}
package handlers_test
import (
	"github.com/gogf/gf/v2/frame/g"
	"github.com/gogf/gf/v2/net/ghttp"
	"github.com/gogf/gf/v2/test/gtest"
	"project/internal/handlers"
	"testing"
)
func TestSoftwareUpdateHandler(t *testing.T) {
	s := g.Server()
	s.BindHandler("/software-update", handlers.SoftwareUpdateHandler)
	s.SetPort(8199)
	s.Start()
	defer s.Shutdown()
	gtest.C(t, func(t *gtest.T) {
		c := g.Client()
		c.SetPrefix("http://127.0.0.1:8199")
		r, err := c.Get("/software-update")
		t.AssertNil(err)
		t.Assert(r.StatusCode, 200)
	})
}
package models_test
import (
	"github.com/gogf/gf/v2/test/gtest"
	"project/internal/models"
	"testing"
)
func TestGetSoftwareUpdates(t *testing.T) {
	gtest.C(t, func(t *gtest.T) {
		updates, err := models.GetSoftwareUpdates()
		t.AssertNil(err)
		t.AssertTrue(len(updates) >= 0)
	})
}
```

创建一个名为 `software_updates` 的表，用于存储软件更新信息：

```
CREATE TABLE software_updates (
    id INT AUTO_INCREMENT PRIMARY KEY,
    update_name VARCHAR(255) NOT NULL,
    version_number VARCHAR(50) NOT NULL,
    update_type VARCHAR(50) NOT NULL,
    update_size INT NOT NULL,
    update_status VARCHAR(50) NOT NULL,
    release_time DATE,
    auto_update VARCHAR(10) NOT NULL,
    force_update VARCHAR(10) NOT NULL
```

```
);
package main
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    s.BindHandler("/", handlers.IndexHandler)
    s.BindHandler("/data-sync", handlers.DataSyncHandler)
    s.Run()
}
toml
[database]
    default.type    = "mysql"
    default.host    = "127.0.0.1"
    default.port    = "3306"
    default.user    = "root"
    default.name    = "your_database_name"
    default.charset = "utf8mb4"
    default.prefix  = ""
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
```

```
}
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
```

```
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    margin-bottom: 20px;
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
    background-size: 16px;
    transition: all 0.3s;
}
.form-select:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
```

```
.switch-container {
    display: flex;
    align-items: center;
}
.switch {
    position: relative;
    display: inline-block;
    width: 50px;
    height: 24px;
    margin-right: 10px;
}
.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}
.slider {
    position: absolute;
    cursor: pointer;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: #ccc;
    transition: .4s;
    border-radius: 24px;
}
.slider:before {
    position: absolute;
    content: "";
    height: 16px;
    width: 16px;
    left: 4px;
    bottom: 4px;
    background-color: white;
    transition: .4s;
    border-radius: 50%;
}
input:checked + .slider {
    background-color: #1890ff;
}
input:checked + .slider:before {
    transform: translateX(26px);
}
.date-picker {
    position: relative;
}
.date-picker input {
    padding-right: 30px;
}
```

```
.date-picker::after {
    content: "  ";
    position: absolute;
    right: 10px;
    top: 50%;
    transform: translateY(-50%);
    pointer-events: none;
}
.table-container {
    margin-top: 20px;
}
table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}
th, td {
    padding: 12px 16px;
    text-align: left;
    border-bottom: 1px solid #eaeaea;
}
th {
    background-color: #fafafa;
    font-weight: 500;
    color: #333;
}
tr:hover {
    background-color: #f5f5f5;
}
.action-btn {
    padding: 4px 8px;
    font-size: 12px;
    margin-right: 5px;
    border-radius: 2px;
    cursor: pointer;
    border: none;
}
.chart-container {
    height: 300px;
    margin-top: 20px;
}
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
    z-index: 1000;
```

```
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
        margin-top: 20px;
        padding-top: 15px;
        border-top: 1px solid #eaeaea;
}
.tips {
        margin-top: 20px;
        padding: 15px;
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
}
```

```
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
/* 新增样式 */
.card {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.card-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 15px;
    padding-bottom: 10px;
    border-bottom: 1px solid #eaeaea;
}
.card-header h3 {
    font-size: 18px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.card-header h3 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 20px;
}
.card-body {
    padding: 10px 0;
}
.progress-container {
    background-color: #f0f0f0;
    border-radius: 4px;
    height: 20px;
    margin-top: 10px;
    overflow: hidden;
}
.progress-bar {
    background-color: #1890ff;
    height: 100%;
```

```
        width: 0;
        text-align: center;
        color: white;
        font-size: 12px;
        line-height: 20px;
}
.info-box {
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
        padding: 15px;
        margin-bottom: 20px;
}
.info-box-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
}
.info-box-content {
        font-size: 14px;
        color: #555;
}
script
document.addEventListener('DOMContentLoaded', function() {
        // 初始化日期选择器
        document.getElementById('startTimePicker').addEventListener('focus', function() {
                this.type = 'datetime-local';
        });
        document.getElementById('endTimePicker').addEventListener('focus', function() {
                this.type = 'datetime-local';
        });
        // 数据控制
        const syncModal = document.getElementById('syncModal');
        const confirmModal = document.getElementById('confirmModal');
        const startSyncBtn = document.getElementById('startSyncBtn');
        const closeSyncModalBtn = document.getElementById('closeSyncModalBtn');
        const closeConfirmModalBtn = document.getElementById('closeConfirmModalBtn');
        const cancelSyncModalBtn = document.getElementById('cancelSyncModalBtn');
        const confirmSyncModalBtn = document.getElementById('confirmSyncModalBtn');
        const cancelDeleteBtn = document.getElementById('cancelDeleteBtn');
        const confirmDeleteBtn = document.getElementById('confirmDeleteBtn');
        startSyncBtn.addEventListener('click', function() {
                syncModal.style.display = 'flex';
        });
        closeSyncModalBtn.addEventListener('click', function() {
                syncModal.style.display = 'none';
        });
        cancelSyncModalBtn.addEventListener('click', function() {
                syncModal.style.display = 'none';
        });
```

```
confirmSyncModalBtn.addEventListener('click', function() {
    alert('同步任务添加成功！');
    syncModal.style.display = 'none';
    const table = document.querySelector('table tbody');
    const newRow = table.insertRow();
    const cells = [
        newRow.insertCell(0),
        newRow.insertCell(1),
        newRow.insertCell(2),
        newRow.insertCell(3),
        newRow.insertCell(4),
        newRow.insertCell(5),
        newRow.insertCell(6),
        newRow.insertCell(7),
        newRow.insertCell(8)
    ];
    cells[0].textContent = '5';
    cells[1].textContent = 'New Server A';
    cells[2].textContent = 'New Server B';
    cells[3].textContent = '10 分钟';
    cells[4].textContent = '增量同步';
    cells[5].textContent = '2024-10-05 08:00';
    cells[6].textContent = '2024-10-05 18:00';
    cells[7].innerHTML = '<span style="color: #52c41a;">待启动</span>';
    cells[8].innerHTML = `
        <button class="action-btn" style="background-color: #1890ff; color: white;">编辑</button>
        <button class="action-btn" style="background-color: #ff4d4f; color: white;">删除</button>
    `;
});
// 表格中的删除按钮点击事件
const deleteButtons = document.querySelectorAll('.action-btn[style*="background-color: #ff4d4f"]');
deleteButtons.forEach(btn => {
    btn.addEventListener('click', function() {
        confirmModal.style.display = 'flex';
    });
});
closeConfirmModalBtn.addEventListener('click', function() {
    confirmModal.style.display = 'none';
});
cancelDeleteBtn.addEventListener('click', function() {
    confirmModal.style.display = 'none';
});
confirmDeleteBtn.addEventListener('click', function() {
    alert('同步任务删除成功！');
    confirmModal.style.display = 'none';
    const targetRow = this.closest('tr');
    if (targetRow) {
        targetRow.remove();
    }
});
```

```
    // 点击数据外部关闭数据
    window.addEventListener('click', function(event) {
        if (event.target === syncModal) {
            syncModal.style.display = 'none';
        }
        if (event.target === confirmModal) {
            confirmModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
    // 刷新进度按钮点击事件
    const refreshProgressBtn = document.getElementById('refreshProgressBtn');
    const progressBar = document.getElementById('progressBar');
    refreshProgressBtn.addEventListener('click', function() {
        let progress = parseInt(progressBar.style.width);
        progress += 10;
        if (progress > 100) {
            progress = 100;
        }
        progressBar.style.width = `${progress}%`;
        progressBar.textContent = `${progress}%`;
    });
});
function initCharts() {
    // 同步状态统计图
    const syncStatusChart = echarts.init(document.getElementById('syncStatusChart'));
    const syncStatusOption = {
        title: {
            text: '同步状态分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item',
            formatter: '{a} <br/>{b}: {c} ({d}%)'
        },
        legend: {
            orient: 'vertical',
            right: 10,
            top: 'center',
            data: ['运行中', '已停止', '待启动']
        },
        series: [
            {
                name: '同步状态',
                type: 'pie',
                radius: ['50%', '70%'],
```

```
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: [
                    { value: 2, name: '运行中', itemStyle: { color: '#52c41a' } },
                    { value: 2, name: '已停止', itemStyle: { color: '#ff4d4f' } },
                    { value: 0, name: '待启动', itemStyle: { color: '#faad14' } }
                ]
            }
        ]
};
syncStatusChart.setOption(syncStatusOption);
// 同步模式统计图
const syncModeChart = echarts.init(document.getElementById('syncModeChart'));
const syncModeOption = {
    title: {
        text: '同步模式分布',
        left: 'center',
        textStyle: {
            color: '#333'
        }
    },
    tooltip: {
        trigger: 'axis',
        axisPointer: {
            type: 'shadow'
        }
    },
    grid: {
        left: '3%',
        right: '4%',
        bottom: '3%',
        containLabel: true
```

```
        },
        xAxis: {
            type: 'value',
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        yAxis: {
            type: 'category',
            data: ['全量同步', '增量同步'],
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        series: [
            {
                name: '数量',
                type: 'bar',
                data: [
                    {
                        value: 2,
                        itemStyle: {
                            color: '#1890ff'
                        }
                    },
                    {
                        value: 2,
                        itemStyle: {
                            color: '#13c2c2'
                        }
                    }
                ],
                barWidth: '40%',
                label: {
                    show: true,
                    position: 'right'
                }
            }
        ]
};
syncModeChart.setOption(syncModeOption);
// 窗口大小变化时重新调整图表大小
window.addEventListener('resize', function() {
    syncStatusChart.resize();
    syncModeChart.resize();
});
```

```
}
package handlers
import (
    "github.com/gogf/gf/net/ghttp"
)
func IndexHandler(r *ghttp.Request) {
    r.Response.WriteTpl("index.html")
}
package handlers
import (
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
)
func DataSyncHandler(r *ghttp.Request) {
    // 从数据库获取同步任务数据
    tasks, err := models.GetDataSyncTasks()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    r.Response.WriteTpl("data_sync.html", g.Map{
        "tasks": tasks,
    })
}
package models
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
)
// DataSyncTask 数据同步任务结构体
type DataSyncTask struct {
    ID          int       `json:"id"`
    DataSource  string `json:"data_source"`
    Target      string `json:"target"`
    Frequency   string `json:"frequency"`
    Mode        string `json:"mode"`
    StartTime   string `json:"start_time"`
    EndTime     string `json:"end_time"`
    Status      string `json:"status"`
}
// GetDataSyncTasks 从数据库获取数据同步任务
func GetDataSyncTasks() ([]*DataSyncTask, error) {
    var tasks []*DataSyncTask
    err := g.DB().Table("data_sync_tasks").Scan(&tasks)
    return tasks, err
}
package utils
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
```

```
)
func InitDB() {
	err := g.DB().InitConfigFromToml("config/config.toml")
	if err != nil {
		g.Log().Fatalf("Failed to initialize database: %v", err)
	}
}
package handlers
import (
	"net/http"
	"net/http/httptest"
	"testing"
	"github.com/gogf/gf/frame/g"
	"github.com/gogf/gf/net/ghttp"
)
func TestDataSyncHandler(t *testing.T) {
	s := g.Server()
	s.BindHandler("/data-sync", DataSyncHandler)
	s.SetPort(8199)
	s.Start()
	defer s.Shutdown()
	req, err := http.NewRequest("GET", "http://127.0.0.1:8199/data-sync", nil)
	if err != nil {
		t.Fatal(err)
	}
	resp := httptest.NewRecorder()
	s.ServeHTTP(resp, req)
	if resp.Code != http.StatusOK {
		t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
	}
}
package models
import (
	"testing"
	"github.com/gogf/gf/database/gdb"
	"github.com/gogf/gf/frame/g"
	"github.com/gogf/gf/test/gtest"
)
func TestGetDataSyncTasks(t *testing.T) {
	gtest.C(t, func(t *gtest.T) {
		tasks, err := GetDataSyncTasks()
		t.AssertNil(err)
		t.AssertTrue(len(tasks) >= 0)
	})
}
```

创建名为 `data_sync_tasks` 的表，用于存储数据同步任务信息：

```
CREATE TABLE `data_sync_tasks` (
	`id` int(11) NOT NULL AUTO_INCREMENT,
	`data_source` varchar(255) NOT NULL,
	`target` varchar(255) NOT NULL,
```

```
    `frequency` varchar(20) NOT NULL,
    `mode` varchar(20) NOT NULL,
    `start_time` datetime NOT NULL,
    `end_time` datetime NOT NULL,
    `status` varchar(20) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
package main
import (
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    s.BindHandler("/", handlers.IndexHandler)
    s.BindHandler("/alarm-records", handlers.AlarmRecordHandler)
    s.Run()
}
toml
[database]
/* 此处复制原型设计中的 CSS 代码 */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
```

```
}
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
```

```
.btn i {
    margin-right: 5px;
}
.form-container {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 20px;
    margin-bottom: 20px;
}
.form-item {
    margin-bottom: 15px;
}
.form-item label {
    display: block;
    margin-bottom: 8px;
    font-size: 14px;
    color: #666;
}
.form-input {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    transition: all 0.3s;
}
.form-input:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
    width: 100%;
    padding: 10px;
    border: 1px solid #d9d9d9;
    border-radius: 4px;
    font-size: 14px;
    appearance: none;
    background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
    background-size: 16px;
    transition: all 0.3s;
}
.form-select:focus {
    border-color: #1890ff;
    outline: none;
    box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
```

```
.switch-container {
    display: flex;
    align-items: center;
}
.switch {
    position: relative;
    display: inline-block;
    width: 50px;
    height: 24px;
    margin-right: 10px;
}
.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}
.slider {
    position: absolute;
    cursor: pointer;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: #ccc;
    transition: .4s;
    border-radius: 24px;
}
.slider:before {
    position: absolute;
    content: "";
    height: 16px;
    width: 16px;
    left: 4px;
    bottom: 4px;
    background-color: white;
    transition: .4s;
    border-radius: 50%;
}
input:checked + .slider {
    background-color: #1890ff;
}
input:checked + .slider:before {
    transform: translateX(26px);
}
.date-picker {
    position: relative;
}
.date-picker input {
    padding-right: 30px;
}
```

```
.date-picker::after {
    content: "  ";
    position: absolute;
    right: 10px;
    top: 50%;
    transform: translateY(-50%);
    pointer-events: none;
}
.table-container {
    margin-top: 20px;
}
table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}
th,
td {
    padding: 12px 16px;
    text-align: left;
    border-bottom: 1px solid #eaeaea;
}
th {
    background-color: #fafafa;
    font-weight: 500;
    color: #333;
}
tr:hover {
    background-color: #f5f5f5;
}
.action-btn {
    padding: 4px 8px;
    font-size: 12px;
    margin-right: 5px;
    border-radius: 2px;
    cursor: pointer;
    border: none;
}
.chart-container {
    height: 300px;
    margin-top: 20px;
}
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.5);
```

```
        z-index: 1000;
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
        margin-top: 20px;
        padding-top: 15px;
        border-top: 1px solid #eaeaea;
}
.tips {
        margin-top: 20px;
        padding: 15px;
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
```

```
}
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
/* 新增样式 */
.btn-warning {
    background-color: #faad14;
    color: white;
}
.btn-warning:hover {
    background-color: #ffc53d;
}
.priority-high {
    color: #ff4d4f;
    font-weight: bold;
}
.priority-medium {
    color: #faad14;
    font-weight: bold;
}
.priority-low {
    color: #52c41a;
    font-weight: bold;
}
.status-resolved {
    color: #52c41a;
}
.status-unresolved {
    color: #ff4d4f;
}
.status-in-progress {
    color: #faad14;
}
script
// 初始化日期选择器
document.addEventListener('DOMContentLoaded', function () {
    // 模拟日期选择器
    document.getElementById('startDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    document.getElementById('endDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    document.getElementById('newAlarmTime').addEventListener('focus', function () {
```

```
        this.type = 'date';
    });
    // 数据控制
    const alarmModal = document.getElementById('alarmModal');
    const confirmModal = document.getElementById('confirmModal');
    const addAlarmBtn = document.getElementById('addAlarmBtn');
    const closeAlarmModalBtn = document.getElementById('closeAlarmModalBtn');
    const closeConfirmModalBtn = document.getElementById('closeConfirmModalBtn');
    const cancelAlarmBtn = document.getElementById('cancelAlarmBtn');
    const confirmAlarmBtn = document.getElementById('confirmAlarmBtn');
    const cancelDeleteBtn = document.getElementById('cancelDeleteBtn');
    const confirmDeleteBtn = document.getElementById('confirmDeleteBtn');
    const clearAlarmBtn = document.getElementById('clearAlarmBtn');
    addAlarmBtn.addEventListener('click', function () {
        alarmModal.style.display = 'flex';
    });
    closeAlarmModalBtn.addEventListener('click', function () {
        alarmModal.style.display = 'none';
    });
    closeConfirmModalBtn.addEventListener('click', function () {
        confirmModal.style.display = 'none';
    });
    cancelAlarmBtn.addEventListener('click', function () {
        alarmModal.style.display = 'none';
    });
    confirmAlarmBtn.addEventListener('click', function () {
        // 获取数据中的输入值
        const newAlarmTime = document.getElementById('newAlarmTime').value;
        const newAlarmType = document.getElementById('newAlarmType').value;
        const newAlarmPriority = document.getElementById('newAlarmPriority').value;
        const newRelatedDevice = document.getElementById('newRelatedDevice').value;
        const newAlarmDescription = document.getElementById('newAlarmDescription').value;
        const newAlarmStatus = document.getElementById('newAlarmStatus').value;
        // 发送请求到后端添加报警记录
        fetch('/add-alarm-record', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({
                alarm_time: newAlarmTime,
                alarm_type: newAlarmType,
                alarm_priority: newAlarmPriority,
                related_device: newRelatedDevice,
                alarm_description: newAlarmDescription,
                alarm_status: newAlarmStatus
            })
        })
          .then(response => response.json())
          .then(data => {
```

```
                if (data.success) {
                    // 关闭数据
                    alarmModal.style.display = 'none';
                    // 清空数据输入值
                    document.getElementById('newAlarmTime').value = '';
                    document.getElementById('newAlarmType').value = '系统故障';
                    document.getElementById('newAlarmPriority').value = '高';
                    document.getElementById('newRelatedDevice').value = '';
                    document.getElementById('newAlarmDescription').value = '';
                    document.getElementById('newAlarmStatus').value = '未解决';
                    // 重新加载图表
                    initCharts();
                }
            });
    });
    clearAlarmBtn.addEventListener('click', function () {
        confirmModal.style.display = 'flex';
    });
    cancelDeleteBtn.addEventListener('click', function () {
        confirmModal.style.display = 'none';
    });
    confirmDeleteBtn.addEventListener('click', function () {
        // 发送请求到后端清空已解决报警记录
        fetch('/clear-resolved-alarms', {
            method: 'POST'
        })
           .then(response => response.json())
           .then(data => {
                if (data.success) {
                    confirmModal.style.display = 'none';
                    // 重新加载图表
                    initCharts();
                }
            });
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function (event) {
        if (event.target === alarmModal) {
            alarmModal.style.display = 'none';
        }
        if (event.target === confirmModal) {
            confirmModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
});
function initCharts() {
    // 获取报警类型统计数据
    fetch('/alarm-type-statistics')
```

```
.then(response => response.json())
.then(data => {
    // 报警类型统计图
    const alarmTypeChart = echarts.init(document.getElementById('alarmTypeChart'));
    const alarmTypeOption = {
        title: {
            text: '报警类型分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item',
            formatter: '{a} <br/>{b}: {c} ({d}%)'
        },
        legend: {
            orient: 'vertical',
            right: 10,
            top: 'center',
            data: ['系统故障', '网络异常', '设备故障']
        },
        series: [
            {
                name: '报警类型',
                type: 'pie',
                radius: ['50%', '70%'],
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: data
            }
        ]
```

```
            };
            alarmTypeChart.setOption(alarmTypeOption);
            // 窗口大小变化时重新调整图表大小
            window.addEventListener('resize', function () {
                alarmTypeChart.resize();
            });
        });
    // 获取报警优先级统计数据
    fetch('/alarm-priority-statistics')
        .then(response => response.json())
        .then(data => {
            // 报警优先级统计图
            const alarmPriorityChart = echarts.init(document.getElementById('alarmPriorityChart'));
            const alarmPriorityOption = {
                title: {
                    text: '报警优先级分布',
                    left: 'center',
                    textStyle: {
                        color: '#333'
                    }
                },
                tooltip: {
                    trigger: 'axis',
                    axisPointer: {
                        type: 'shadow'
                    }
                },
                grid: {
                    left: '3%',
                    right: '4%',
                    bottom: '3%',
                    containLabel: true
                },
                xAxis: {
                    type: 'value',
                    axisLine: {
                        lineStyle: {
                            color: '#999'
                        }
                    }
                },
                yAxis: {
                    type: 'category',
                    data: ['高', '中', '低'],
                    axisLine: {
                        lineStyle: {
                            color: '#999'
                        }
                    }
                },
```

```
                series: [
                    {
                        name: '数量',
                        type: 'bar',
                        data: data,
                        barWidth: '40%',
                        label: {
                            show: true,
                            position: 'right'
                        }
                    }
                ]
            };
            alarmPriorityChart.setOption(alarmPriorityOption);
            // 窗口大小变化时重新调整图表大小
            window.addEventListener('resize', function () {
                alarmPriorityChart.resize();
            });
        });
}
package handlers
import (
    "encoding/json"
    "net/http"
    "project/internal/models"
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
)
// AlarmRecordHandler 处理异常报警记录页面请求
func AlarmRecordHandler(r *ghttp.Request) {
    // 获取报警记录数据
    alarmRecords, err := models.GetAlarmRecords()
    if err != nil {
        r.Response.WriteStatus(http.StatusInternalServerError)
        return
    }
    // 渲染页面
    r.Response.WriteTpl("alarm_records.html", g.Map{
        "alarmRecords": alarmRecords,
    })
}
// AddAlarmRecordHandler 处理新增报警记录请求
func AddAlarmRecordHandler(r *ghttp.Request) {
    var alarmRecord models.AlarmRecord
    if err := r.Parse(&alarmRecord); err != nil {
        r.Response.WriteJson(g.Map{
            "success": false,
            "message": err.Error(),
        })
        return
```

```
	}
	if err := models.AddAlarmRecord(alarmRecord); err != nil {
		r.Response.WriteJson(g.Map{
			"success": false,
			"message": err.Error(),
		})
		return
	}
	r.Response.WriteJson(g.Map{
		"success": true,
	})
}
// ClearResolvedAlarmsHandler 处理清空已解决报警记录请求
func ClearResolvedAlarmsHandler(r *ghttp.Request) {
	if err := models.ClearResolvedAlarms(); err != nil {
		r.Response.WriteJson(g.Map{
			"success": false,
			"message": err.Error(),
		})
		return
	}
	r.Response.WriteJson(g.Map{
		"success": true,
	})
}
// AlarmTypeStatisticsHandler 处理报警类型统计数据请求
func AlarmTypeStatisticsHandler(r *ghttp.Request) {
	statistics, err := models.GetAlarmTypeStatistics()
	if err != nil {
		r.Response.WriteStatus(http.StatusInternalServerError)
		return
	}
	r.Response.WriteJson(statistics)
}
// AlarmPriorityStatisticsHandler 处理报警优先级统计数据请求
func AlarmPriorityStatisticsHandler(r *ghttp.Request) {
	statistics, err := models.GetAlarmPriorityStatistics()
	if err != nil {
		r.Response.WriteStatus(http.StatusInternalServerError)
		return
	}
	r.Response.WriteJson(statistics)
}
package models
import (
	"database/sql"
	"fmt"
	_ "github.com/go-sql-driver/mysql"
	"github.com/gogf/gf/v2/frame/g"
)
```

```
// AlarmRecord 定义报警记录结构体
type AlarmRecord struct {
    AlarmTime        string `json:"alarm_time"`
    AlarmType        string `json:"alarm_type"`
    AlarmPriority    string `json:"alarm_priority"`
    AlarmStatus      string `json:"alarm_status"`
    RelatedDevice    string `json:"related_device"`
    AlarmDescription string `json:"alarm_description"`
}
// GetAlarmRecords 获取报警记录数据
func GetAlarmRecords() ([]AlarmRecord, error) {
    db, err := getDB()
    if err != nil {
        return nil, err
    }
    defer db.Close()
    rows, err := db.Query("SELECT alarm_time, alarm_type, alarm_priority, alarm_status, related_device, alarm_description FROM alarm_records")
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var alarmRecords []AlarmRecord
    for rows.Next() {
        var record AlarmRecord
        if err := rows.Scan(&record.AlarmTime, &record.AlarmType, &record.AlarmPriority, &record.AlarmStatus, &record.RelatedDevice, &record.AlarmDescription); err != nil {
            return nil, err
        }
        alarmRecords = append(alarmRecords, record)
    }
    return alarmRecords, nil
}
// AddAlarmRecord 新增报警记录
func AddAlarmRecord(record AlarmRecord) error {
    db, err := getDB()
    if err != nil {
        return err
    }
    defer db.Close()
    _, err = db.Exec("INSERT INTO alarm_records (alarm_time, alarm_type, alarm_priority, alarm_status, related_device, alarm_description) VALUES (?,?,?,?,?,?)",
        record.AlarmTime, record.AlarmType, record.AlarmPriority, record.AlarmStatus, record.RelatedDevice, record.AlarmDescription)
    return err
}
// ClearResolvedAlarms 清空已解决报警记录
func ClearResolvedAlarms() error {
    db, err := getDB()
    if err != nil {
```

```
        return err
    }
    defer db.Close()
    _, err = db.Exec("DELETE FROM alarm_records WHERE alarm_status = '已解决'")
    return err
}
// GetAlarmTypeStatistics 获取报警类型统计数据
func GetAlarmTypeStatistics() ([]interface{}, error) {
    db, err := getDB()
    if err != nil {
        return nil, err
    }
    defer db.Close()
    rows, err := db.Query("SELECT alarm_type, COUNT(*) as count FROM alarm_records GROUP BY alarm_type")
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var statistics []interface{}
    for rows.Next() {
        var alarmType string
        var count int
        if err := rows.Scan(&alarmType, &count); err != nil {
            return nil, err
        }
        color := ""
        switch alarmType {
        case "系统故障":
            color = "#1890ff"
        case "网络异常":
            color = "#faad14"
        case "设备故障":
            color = "#52c41a"
        }
        statistics = append(statistics, g.Map{
            "value": count,
            "name":   alarmType,
            "itemStyle": g.Map{
                "color": color,
            },
        })
    }
    return statistics, nil
}
// GetAlarmPriorityStatistics 获取报警优先级统计数据
func GetAlarmPriorityStatistics() ([]interface{}, error) {
    db, err := getDB()
    if err != nil {
        return nil, err
```

```
    }
    defer db.Close()
    rows, err := db.Query("SELECT alarm_priority, COUNT(*) as count FROM alarm_records GROUP BY alarm_priority")
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var statistics []interface{}
    for rows.Next() {
        var alarmPriority string
        var count int
        if err := rows.Scan(&alarmPriority, &count); err != nil {
            return nil, err
        }
        color := ""
        switch alarmPriority {
        case "高":
            color = "#ff4d4f"
        case "中":
            color = "#faad14"
        case "低":
            color = "#52c41a"
        }
        statistics = append(statistics, g.Map{
            "value": count,
            "itemStyle": g.Map{
                "color": color,
            },
        })
    }
    return statistics, nil
}
// getDB 获取数据库连接
func getDB() (*sql.DB, error) {
    link, err := g.Cfg().Get(g.Context(), "database.link")
    if err != nil {
        return nil, err
    }
    db, err := sql.Open("mysql", link.String())
    if err != nil {
        return nil, err
    }
    if err := db.Ping(); err != nil {
        return nil, err
    }
    return db, nil
}
package utils
import (
```

```
    "database/sql"
    "github.com/gogf/gf/v2/frame/g"
    _ "github.com/go-sql-driver/mysql"
)
// GetDB 获取数据库连接
func GetDB() (*sql.DB, error) {
    link, err := g.Cfg().Get(g.Context(), "database.link")
    if err != nil {
        return nil, err
    }
    db, err := sql.Open("mysql", link.String())
    if err != nil {
        return nil, err
    }
    if err := db.Ping(); err != nil {
        return nil, err
    }
    return db, nil
}
package utils
import (
    "github.com/gogf/gf/v2/frame/g"
)
// GenerateEChartsOption 生成 ECharts 配置选项
func GenerateEChartsOption(data []interface{}) g.Map {
    return g.Map{
        "title": g.Map{
            "text": "报警类型分布",
            "left": "center",
            "textStyle": g.Map{
                "color": "#333",
            },
        },
        "tooltip": g.Map{
            "trigger": "item",
            "formatter": "{a} <br/>{b}: {c} ({d}%)",
        },
        "legend": g.Map{
            "orient": "vertical",
            "right": 10,
            "top": "center",
            "data": []string{"系统故障", "网络异常", "设备故障"},
        },
        "series": []g.Map{
            {
                "name": "报警类型",
                "type": "pie",
                "radius": []string{"50%", "70%"},
                "avoidLabelOverlap": false,
                "itemStyle": g.Map{
```

```
                        "borderRadius": 10,
                        "borderColor": "#fff",
                        "borderWidth": 2,
                    },
                    "label": g.Map{
                        "show": false,
                        "position": "center",
                    },
                    "emphasis": g.Map{
                        "label": g.Map{
                            "show": true,
                            "fontSize": "18",
                            "fontWeight": "bold",
                        },
                    },
                    "labelLine": g.Map{
                        "show": false,
                    },
                    "data": data,
                },
            },
        }
}
package handlers
import (
    "net/http"
    "net/http/httptest"
    "testing"
    "github.com/gogf/gf/v2/frame/g"
    "github.com/gogf/gf/v2/net/ghttp"
    "project/internal/models"
)
func TestAlarmRecordHandler(t *testing.T) {
    s := g.Server()
    s.BindHandler("/alarm-records", AlarmRecordHandler)
    s.SetPort(8199)
    s.Start()
    defer s.Shutdown()
    req := httptest.NewRequest("GET", "/alarm-records", nil)
    resp := httptest.NewRecorder()
    s.ServeHTTP(resp, req)
    if resp.Code != http.StatusOK {
        t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
    }
}
func TestAddAlarmRecordHandler(t *testing.T) {
    s := g.Server()
    s.BindHandler("/add-alarm-record", AddAlarmRecordHandler)
    s.SetPort(8199)
    s.Start()
```

```
	defer s.Shutdown()
	record := models.AlarmRecord{
		AlarmTime: "2024-10-01 10:00",
		AlarmType: "系统故障",
		AlarmPriority: "高",
		AlarmStatus: "未解决",
		RelatedDevice: "服务器 01",
		AlarmDescription: "服务器 CPU 使用率过高",
	}
	body, err := gjson.Marshal(record)
	if err != nil {
		t.Fatal(err)
	}
	req := httptest.NewRequest("POST", "/add-alarm-record", bytes.NewReader(body))
	req.Header.Set("Content-Type", "application/json")
	resp := httptest.NewRecorder()
	s.ServeHTTP(resp, req)
	if resp.Code != http.StatusOK {
		t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
	}
}
func TestClearResolvedAlarmsHandler(t *testing.T) {
	s := g.Server()
	s.BindHandler("/clear-resolved-alarms", ClearResolvedAlarmsHandler)
	s.SetPort(8199)
	s.Start()
	defer s.Shutdown()
	req := httptest.NewRequest("POST", "/clear-resolved-alarms", nil)
	resp := httptest.NewRecorder()
	s.ServeHTTP(resp, req)
	if resp.Code != http.StatusOK {
		t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
	}
}
func TestAlarmTypeStatisticsHandler(t *testing.T) {
	s := g.Server()
	s.BindHandler("/alarm-type-statistics", AlarmTypeStatisticsHandler)
	s.SetPort(8199)
	s.Start()
	defer s.Shutdown()
	req := httptest.NewRequest("GET", "/alarm-type-statistics", nil)
	resp := httptest.NewRecorder()
	s.ServeHTTP(resp, req)
	if resp.Code != http.StatusOK {
		t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
	}
}
func TestAlarmPriorityStatisticsHandler(t *testing.T) {
	s := g.Server()
	s.BindHandler("/alarm-priority-statistics", AlarmPriorityStatisticsHandler)
```

```
    s.SetPort(8199)
    s.Start()
    defer s.Shutdown()
    req := httptest.NewRequest("GET", "/alarm-priority-statistics", nil)
    resp := httptest.NewRecorder()
    s.ServeHTTP(resp, req)
    if resp.Code != http.StatusOK {
        t.Errorf("Expected status code %d, but got %d", http.StatusOK, resp.Code)
    }
}
package models
import (
    "testing"
    "github.com/gogf/gf/v2/test/gtest"
)
func TestGetAlarmRecords(t *testing.T) {
    gtest.C(t, func(t *gtest.T) {
        records, err := GetAlarmRecords()
        t.AssertNil(err)
        t.AssertGreater(len(records), 0)
    })
}
func TestAddAlarmRecord(t *testing.T) {
    gtest.C(t, func(t *gtest.T) {
        record := AlarmRecord{
            AlarmTime: "2024-10-01 10:00",
            AlarmType: "系统故障",
            AlarmPriority: "高",
            AlarmStatus: "未解决",
            RelatedDevice: "服务器 01",
            AlarmDescription: "服务器 CPU 使用率过高",
        }
        err := AddAlarmRecord(record)
        t.AssertNil(err)
    })
}
package main
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    s.BindHandler("/", handlers.Index)
    s.BindHandler("/performance-analysis", handlers.PerformanceAnalysis)
    s.Run()
}
toml
[database]
```

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
}
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
```

```
        border: none;
        transition: all 0.3s;
        display: flex;
        align-items: center;
        justify-content: center;
}
.btn-primary {
        background-color: #1890ff;
        color: white;
}
.btn-primary:hover {
        background-color: #40a9ff;
}
.btn-default {
        background-color: #f0f0f0;
        color: #333;
        border: 1px solid #d9d9d9;
}
.btn-default:hover {
        background-color: #e6e6e6;
}
.btn-danger {
        background-color: #ff4d4f;
        color: white;
}
.btn-danger:hover {
        background-color: #ff7875;
}
.btn i {
        margin-right: 5px;
}
.form-container {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        gap: 20px;
        margin-bottom: 20px;
}
.form-item {
        margin-bottom: 15px;
}
.form-item label {
        display: block;
        margin-bottom: 8px;
        font-size: 14px;
        color: #666;
}
.form-input {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
```

```
        border-radius: 4px;
        font-size: 14px;
        transition: all 0.3s;
}
.form-input:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
        border-radius: 4px;
        font-size: 14px;
        appearance: none;
        background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
        background-size: 16px;
        transition: all 0.3s;
}
.form-select:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
        display: flex;
        align-items: center;
}
.switch {
        position: relative;
        display: inline-block;
        width: 50px;
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
```

```
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
.table-container {
        margin-top: 20px;
}
table {
        width: 100%;
        border-collapse: collapse;
        font-size: 14px;
}
th, td {
        padding: 12px 16px;
        text-align: left;
        border-bottom: 1px solid #eaeaea;
}
th {
```

```
        background-color: #fafafa;
        font-weight: 500;
        color: #333;
}
tr:hover {
        background-color: #f5f5f5;
}
.action-btn {
        padding: 4px 8px;
        font-size: 12px;
        margin-right: 5px;
        border-radius: 2px;
        cursor: pointer;
        border: none;
}
.chart-container {
        height: 300px;
        margin-top: 20px;
}
.modal {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        background-color: rgba(0, 0, 0, 0.5);
        z-index: 1000;
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
```

```
}
.close-btn {
    background: none;
    border: none;
    font-size: 20px;
    cursor: pointer;
    color: #999;
}
.modal-footer {
    display: flex;
    justify-content: flex-end;
    gap: 10px;
    margin-top: 20px;
    padding-top: 15px;
    border-top: 1px solid #eaeaea;
}
.tips {
    margin-top: 20px;
    padding: 15px;
    background-color: #f0f9ff;
    border-left: 4px solid #1890ff;
    border-radius: 4px;
}
.tips-title {
    font-weight: 600;
    margin-bottom: 8px;
    color: #1890ff;
}
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
script
document.addEventListener('DOMContentLoaded', function() {
    // 初始化日期选择器
    document.getElementById('startDate').addEventListener('focus', function() {
        this.type = 'date';
    });
    document.getElementById('endDate').addEventListener('focus', function() {
        this.type = 'date';
    });
    // 数据控制
    const detailModal = document.getElementById('detailModal');
    const closeDetailModalBtn = document.getElementById('closeDetailModalBtn');
    const closeDetailBtn = document.getElementById('closeDetailBtn');
```

```
    const detailBtns = document.querySelectorAll('.action-btn');
    detailBtns.forEach(btn => {
        btn.addEventListener('click', function() {
            const row = this.closest('tr');
            const cells = row.cells;
            document.getElementById('detailTime').value = cells[0].textContent;
            document.getElementById('detailModule').value = cells[1].textContent;
            document.getElementById('detailUser').value = cells[2].textContent;
            document.getElementById('detailResponseTime').value = cells[3].textContent;
            document.getElementById('detailThroughput').value = cells[4].textContent;
            document.getElementById('detailErrorRate').value = cells[5].textContent;
            document.getElementById('detailDescription').value = '这是该条记录的详细描述信息。';
            detailModal.style.display = 'flex';
        });
    });
    closeDetailModalBtn.addEventListener('click', function() {
        detailModal.style.display = 'none';
    });
    closeDetailBtn.addEventListener('click', function() {
        detailModal.style.display = 'none';
    });
    window.addEventListener('click', function(event) {
        if (event.target === detailModal) {
            detailModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
});
function initCharts() {
    // 从服务器获取数据
    fetch('/performance-analysis-data')
      .then(response => response.json())
      .then(data => {
            // 响应时间折线图
            const responseTimeChart = echarts.init(document.getElementById('responseTimeChart'));
            const responseTimeOption = {
                title: {
                    text: '响应时间变化趋势',
                    left: 'center',
                    textStyle: {
                        color: '#333'
                    }
                },
                tooltip: {
                    trigger: 'axis'
                },
                xAxis: {
                    type: 'category',
                    data: data.responseTime.xAxis,
```

```
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        yAxis: {
            type: 'value',
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        series: [
            {
                name: '响应时间',
                type: 'line',
                data: data.responseTime.series,
                itemStyle: {
                    color: '#1890ff'
                }
            }
        ]
};
responseTimeChart.setOption(responseTimeOption);
// 吞吐量柱状图
const throughputChart = echarts.init(document.getElementById('throughputChart'));
const throughputOption = {
        title: {
            text: '吞吐量分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'axis',
            axisPointer: {
                type: 'shadow'
            }
        },
        xAxis: {
            type: 'category',
            data: data.throughput.xAxis,
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
```

```
        },
        yAxis: {
            type: 'value',
            axisLine: {
                lineStyle: {
                    color: '#999'
                }
            }
        },
        series: [
            {
                name: '吞吐量',
                type: 'bar',
                data: data.throughput.series,
                itemStyle: {
                    color: '#52c41a'
                },
                barWidth: '40%',
                label: {
                    show: true,
                    position: 'top'
                }
            }
        ]
};
throughputChart.setOption(throughputOption);
// 错误率饼图
const errorRateChart = echarts.init(document.getElementById('errorRateChart'));
const errorRateOption = {
        title: {
            text: '错误率分布',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item',
            formatter: '{a} <br/>{b}: {c} ({d}%)'
        },
        legend: {
            orient: 'vertical',
            right: 10,
            top: 'center',
            data: data.errorRate.legend
        },
        series: [
            {
                name: '错误率',
                type: 'pie',
```

```
                radius: ['50%', '70%'],
                avoidLabelOverlap: false,
                itemStyle: {
                    borderRadius: 10,
                    borderColor: '#fff',
                    borderWidth: 2
                },
                label: {
                    show: false,
                    position: 'center'
                },
                emphasis: {
                    label: {
                        show: true,
                        fontSize: '18',
                        fontWeight: 'bold'
                    }
                },
                labelLine: {
                    show: false
                },
                data: data.errorRate.series
            }
        ]
    };
    errorRateChart.setOption(errorRateOption);
    // 模块性能雷达图
    const modulePerformanceChart = echarts.init(document.getElementById('modulePerformanceChart'));
    const modulePerformanceOption = {
        title: {
            text: '模块性能综合评估',
            left: 'center',
            textStyle: {
                color: '#333'
            }
        },
        tooltip: {
            trigger: 'item'
        },
        radar: {
            indicator: data.modulePerformance.indicator
        },
        series: data.modulePerformance.series
    };
    modulePerformanceChart.setOption(modulePerformanceOption);
    window.addEventListener('resize', function() {
        responseTimeChart.resize();
        throughputChart.resize();
        errorRateChart.resize();
```

```
                modulePerformanceChart.resize();
            });
        });
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
)
func PerformanceAnalysis(r *ghttp.Request) {
    // 从数据库获取数据
    data, err := models.GetPerformanceAnalysisData()
    if err != nil {
        r.Response.WriteStatus(500)
        return
    }
    // 返回数据给前端
    r.Response.WriteJson(data)
}
package models
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
)
// 定义性能分析数据结构
type PerformanceAnalysisData struct {
    ResponseTime struct {
        XAxis  []string
        Series []int
    }
    Throughput struct {
        XAxis  []string
        Series []int
    }
    ErrorRate struct {
        Legend []string
        Series []map[string]interface{}
    }
    ModulePerformance struct {
        Indicator []map[string]interface{}
        Series    []map[string]interface{}
    }
}
// 获取性能分析数据
func GetPerformanceAnalysisData() (*PerformanceAnalysisData, error) {
    db := g.DB()
    var data PerformanceAnalysisData
    // 查询响应时间数据
    responseTimeResult, err := db.Table("performance_analysis").Fields("analysis_time,
```

```
response_time").Order("analysis_time").All()
    if err != nil {
        return nil, err
    }
    for _, row := range responseTimeResult {
        data.ResponseTime.XAxis = append(data.ResponseTime.XAxis, row.GetString("analysis_time"))
        data.ResponseTime.Series = append(data.ResponseTime.Series, row.GetInt("response_time"))
    }
    // 查询吞吐量数据
    throughputResult, err := db.Table("performance_analysis").Fields("module_name,
throughput").Group("module_name").All()
    if err != nil {
        return nil, err
    }
    for _, row := range throughputResult {
        data.Throughput.XAxis = append(data.Throughput.XAxis, row.GetString("module_name"))
        data.Throughput.Series = append(data.Throughput.Series, row.GetInt("throughput"))
    }
    // 查询错误率数据
    errorRateResult, err := db.Table("performance_analysis").Fields("module_name,
error_rate").Group("module_name").All()
    if err != nil {
        return nil, err
    }
    for _, row := range errorRateResult {
        data.ErrorRate.Legend = append(data.ErrorRate.Legend, row.GetString("module_name"))
        data.ErrorRate.Series = append(data.ErrorRate.Series, map[string]interface{}{
            "value":     row.GetFloat64("error_rate"),
            "name":      row.GetString("module_name"),
            "itemStyle": map[string]string{"color": "#ff4d4f"},
        })
    }
    // 查询模块性能数据
    modulePerformanceResult, err := db.Table("performance_analysis").Fields("module_name,
response_time, throughput, error_rate, stability, compatibility").All()
    if err != nil {
        return nil, err
    }
    data.ModulePerformance.Indicator = []map[string]interface{}{
        {"name": "响应时间", "max": 500},
        {"name": "吞吐量", "max": 1500},
        {"name": "错误率", "max": 0.5},
        {"name": "稳定性", "max": 100},
        {"name": "兼容性", "max": 100},
    }
    for _, row := range modulePerformanceResult {
        data.ModulePerformance.Series = append(data.ModulePerformance.Series, map[string]interface{}{
            "name": row.GetString("module_name"),
            "type": "radar",
            "data": []interface{}{
```

```
                map[string]interface{}{
                    "value": []interface{}{
                        row.GetInt("response_time"),
                        row.GetInt("throughput"),
                        row.GetFloat64("error_rate"),
                        row.GetInt("stability"),
                        row.GetInt("compatibility"),
                    },
                    "name": row.GetString("module_name"),
                },
            },
        })
    }
    return &data, nil
}
package utils
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/os/gcfg"
)
func InitDB() error {
    cfg := gcfg.New()
    err := cfg.SetFileName("config.toml")
    if err != nil {
        return err
    }
    err = g.DB().SetConfig(cfg.GetMap("database"))
    if err != nil {
        return err
    }
    return nil
}
package utils
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
)
// 初始化 ECharts 数据
func InitEChartsData(r *ghttp.Request) {
    r.Response.WriteJson(g.Map{})
}
```

创建一个名为 `performance_analysis` 的表，用于存储性能分析数据：

```
CREATE TABLE `performance_analysis` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `analysis_time` datetime NOT NULL,
    `module_name` varchar(255) NOT NULL,
    `user_id` varchar(255) NOT NULL,
    `response_time` int(11) NOT NULL,
    `throughput` int(11) NOT NULL,
    `error_rate` float NOT NULL,
```

```
    `stability` int(11) NOT NULL,
    `compatibility` int(11) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
package main
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/handlers"
)
func main() {
    s := g.Server()
    // 注册路由
    s.BindHandler("/", handlers.IndexHandler)
    s.BindHandler("/maintenance_log", handlers.MaintenanceLogHandler)
    s.Run()
}
toml
[database]
default.type = "mysql"
/* 样式代码与原型设计中的 CSS 一致 */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Microsoft YaHei", sans-serif;
}
body {
    background-color: #f5f5f5;
    color: #333;
    width: 1710px;
    margin: 0 auto;
    padding: 20px;
    background: linear-gradient(135deg, #f0f2f5 0%, #dfe6e9 100%);
}
.container {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
    padding: 20px;
    margin-bottom: 20px;
}
.header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 15px;
    border-bottom: 1px solid #eaeaea;
}
```

```
.header h2 {
    font-size: 22px;
    color: #333;
    font-weight: 600;
    display: flex;
    align-items: center;
}
.header h2 i {
    margin-right: 10px;
    color: #1890ff;
    font-size: 24px;
}
.btn-group {
    display: flex;
    gap: 10px;
}
.btn {
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    border: none;
    transition: all 0.3s;
    display: flex;
    align-items: center;
    justify-content: center;
}
.btn-primary {
    background-color: #1890ff;
    color: white;
}
.btn-primary:hover {
    background-color: #40a9ff;
}
.btn-default {
    background-color: #f0f0f0;
    color: #333;
    border: 1px solid #d9d9d9;
}
.btn-default:hover {
    background-color: #e6e6e6;
}
.btn-danger {
    background-color: #ff4d4f;
    color: white;
}
.btn-danger:hover {
    background-color: #ff7875;
}
.btn i {
```

```
        margin-right: 5px;
}
.form-container {
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 20px;
        margin-bottom: 20px;
}
.form-item {
        margin-bottom: 15px;
}
.form-item label {
        display: block;
        margin-bottom: 8px;
        font-size: 14px;
        color: #666;
}
.form-input {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
        border-radius: 4px;
        font-size: 14px;
        transition: all 0.3s;
}
.form-input:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.form-select {
        width: 100%;
        padding: 10px;
        border: 1px solid #d9d9d9;
        border-radius: 4px;
        font-size: 14px;
        appearance: none;
        background: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%23777777' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E") no-repeat right 10px center;
        background-size: 16px;
        transition: all 0.3s;
}
.form-select:focus {
        border-color: #1890ff;
        outline: none;
        box-shadow: 0 0 0 2px rgba(24, 144, 255, 0.2);
}
.switch-container {
```

```
        display: flex;
        align-items: center;
}
.switch {
        position: relative;
        display: inline-block;
        width: 50px;
        height: 24px;
        margin-right: 10px;
}
.switch input {
        opacity: 0;
        width: 0;
        height: 0;
}
.slider {
        position: absolute;
        cursor: pointer;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background-color: #ccc;
        transition: .4s;
        border-radius: 24px;
}
.slider:before {
        position: absolute;
        content: "";
        height: 16px;
        width: 16px;
        left: 4px;
        bottom: 4px;
        background-color: white;
        transition: .4s;
        border-radius: 50%;
}
input:checked + .slider {
        background-color: #1890ff;
}
input:checked + .slider:before {
        transform: translateX(26px);
}
.date-picker {
        position: relative;
}
.date-picker input {
        padding-right: 30px;
}
.date-picker::after {
```

```
        content: "    ";
        position: absolute;
        right: 10px;
        top: 50%;
        transform: translateY(-50%);
        pointer-events: none;
}
.table-container {
        margin-top: 20px;
}
table {
        width: 100%;
        border-collapse: collapse;
        font-size: 14px;
}
th,
td {
        padding: 12px 16px;
        text-align: left;
        border-bottom: 1px solid #eaeaea;
}
th {
        background-color: #fafafa;
        font-weight: 500;
        color: #333;
}
tr:hover {
        background-color: #f5f5f5;
}
.action-btn {
        padding: 4px 8px;
        font-size: 12px;
        margin-right: 5px;
        border-radius: 2px;
        cursor: pointer;
        border: none;
}
.chart-container {
        height: 300px;
        margin-top: 20px;
}
.modal {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        background-color: rgba(0, 0, 0, 0.5);
        z-index: 1000;
```

```
        justify-content: center;
        align-items: center;
}
.modal-content {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 600px;
        max-width: 90%;
        box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
}
.modal-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 20px;
        padding-bottom: 10px;
        border-bottom: 1px solid #eaeaea;
}
.modal-title {
        font-size: 18px;
        font-weight: 600;
}
.close-btn {
        background: none;
        border: none;
        font-size: 20px;
        cursor: pointer;
        color: #999;
}
.modal-footer {
        display: flex;
        justify-content: flex-end;
        gap: 10px;
        margin-top: 20px;
        padding-top: 15px;
        border-top: 1px solid #eaeaea;
}
.tips {
        margin-top: 20px;
        padding: 15px;
        background-color: #f0f9ff;
        border-left: 4px solid #1890ff;
        border-radius: 4px;
}
.tips-title {
        font-weight: 600;
        margin-bottom: 8px;
        color: #1890ff;
}
```

```
.tips-content {
    font-size: 14px;
    color: #555;
}
.grid-container {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
script
document.addEventListener('DOMContentLoaded', function () {
    // 初始化日期选择器
    document.getElementById('startDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    document.getElementById('endDate').addEventListener('focus', function () {
        this.type = 'date';
    });
    document.getElementById('logDateTime').addEventListener('focus', function () {
        this.type = 'datetime-local';
    });
    // 数据控制
    const logModal = document.getElementById('logModal');
    const confirmDeleteModal = document.getElementById('confirmDeleteModal');
    const addLogBtn = document.getElementById('addLogBtn');
    const closeLogModalBtn = document.getElementById('closeLogModalBtn');
    const closeConfirmDeleteModalBtn = document.getElementById('closeConfirmDeleteModalBtn');
    const cancelLogBtn = document.getElementById('cancelLogBtn');
    const confirmLogBtn = document.getElementById('confirmLogBtn');
    const cancelDeleteLogBtn = document.getElementById('cancelDeleteLogBtn');
    const confirmDeleteLogBtn = document.getElementById('confirmDeleteLogBtn');
    addLogBtn.addEventListener('click', function () {
        logModal.style.display = 'flex';
    });
    closeLogModalBtn.addEventListener('click', function () {
        logModal.style.display = 'none';
    });
    cancelLogBtn.addEventListener('click', function () {
        logModal.style.display = 'none';
    });
    confirmLogBtn.addEventListener('click', function () {
        alert('日志添加成功！');
        logModal.style.display = 'none';
        fetch('/maintenance_log')
            .then(response => response.json())
            .then(data => {
                // 更新表格数据
                updateTable(data.logs);
                // 更新图表数据
                updateCharts(data.operationTypeStats, data.operationResultStats);
```

```
            });
    });
    // 表格中的删除按钮点击事件
    const deleteLogButtons = document.querySelectorAll('.action-btn[style*="background-color: #ff4d4f"]');
    deleteLogButtons.forEach(btn => {
        btn.addEventListener('click', function () {
            confirmDeleteModal.style.display = 'flex';
        });
    });
    closeConfirmDeleteModalBtn.addEventListener('click', function () {
        confirmDeleteModal.style.display = 'none';
    });
    cancelDeleteLogBtn.addEventListener('click', function () {
        confirmDeleteModal.style.display = 'none';
    });
    confirmDeleteLogBtn.addEventListener('click', function () {
        alert('日志删除成功！');
        confirmDeleteModal.style.display = 'none';
        fetch('/maintenance_log')
            .then(response => response.json())
            .then(data => {
                // 更新表格数据
                updateTable(data.logs);
                // 更新图表数据
                updateCharts(data.operationTypeStats, data.operationResultStats);
            });
    });
    // 点击数据外部关闭数据
    window.addEventListener('click', function (event) {
        if (event.target === logModal) {
            logModal.style.display = 'none';
        }
        if (event.target === confirmDeleteModal) {
            confirmDeleteModal.style.display = 'none';
        }
    });
    // 初始化图表
    initCharts();
    // 页面加载时获取数据并更新表格和图表
    fetch('/maintenance_log')
        .then(response => response.json())
        .then(data => {
            // 更新表格数据
            updateTable(data.logs);
            // 更新图表数据
            updateCharts(data.operationTypeStats, data.operationResultStats);
        });
});
function initCharts() {
```

```
// 操作类型分布统计图
const operationTypeChart = echarts.init(document.getElementById('operationTypeChart'));
const operationTypeOption = {
    title: {
        text: '操作类型分布',
        left: 'center',
        textStyle: {
            color: '#333'
        }
    },
    tooltip: {
        trigger: 'item',
        formatter: '{a} <br/>{b}: {c} ({d}%)'
    },
    legend: {
        orient: 'vertical',
        right: 10,
        top: 'center',
        data: ['系统更新', '设备维护', '数据备份']
    },
    series: [
        {
            name: '操作类型',
            type: 'pie',
            radius: ['50%', '70%'],
            avoidLabelOverlap: false,
            itemStyle: {
                borderRadius: 10,
                borderColor: '#fff',
                borderWidth: 2
            },
            label: {
                show: false,
                position: 'center'
            },
            emphasis: {
                label: {
                    show: true,
                    fontSize: '18',
                    fontWeight: 'bold'
                }
            },
            labelLine: {
                show: false
            },
            data: []
        }
    ]
};
operationTypeChart.setOption(operationTypeOption);
```

```
// 操作结果分布统计图
const operationResultChart = echarts.init(document.getElementById('operationResultChart'));
const operationResultOption = {
    title: {
        text: '操作结果分布',
        left: 'center',
        textStyle: {
            color: '#333'
        }
    },
    tooltip: {
        trigger: 'axis',
        axisPointer: {
            type: 'shadow'
        }
    },
    grid: {
        left: '3%',
        right: '4%',
        bottom: '3%',
        containLabel: true
    },
    xAxis: {
        type: 'value',
        axisLine: {
            lineStyle: {
                color: '#999'
            }
        }
    },
    yAxis: {
        type: 'category',
        data: ['成功', '失败'],
        axisLine: {
            lineStyle: {
                color: '#999'
            }
        }
    },
    series: [
        {
            name: '数量',
            type: 'bar',
            data: [],
            barWidth: '40%',
            label: {
                show: true,
                position: 'right'
            }
        }
```

```
            ]
        };
        operationResultChart.setOption(operationResultOption);
        window.operationTypeChart = operationTypeChart;
        window.operationResultChart = operationResultChart;
        // 窗口大小变化时重新调整图表大小
        window.addEventListener('resize', function () {
            operationTypeChart.resize();
            operationResultChart.resize();
        });
}
function updateTable(logs) {
        const tableBody = document.querySelector('table tbody');
        tableBody.innerHTML = '';
        logs.forEach(log => {
            const row = document.createElement('tr');
            row.innerHTML = `
                <td>${log.OperationType}</td>
                <td>${log.Operator}</td>
                <td>${log.OperationTime}</td>
                <td><span style="color: ${log.OperationResult === '成功' ? '#52c41a' :
'#ff4d4f'}">${log.OperationResult}</span></td>
                <td>${log.DeviceName}</td>
                <td><span style="color: ${log.LogStatus === '启用' ? '#52c41a' :
'#ff4d4f'}">${log.LogStatus}</span></td>
                <td>
                    <button class="action-btn" style="background-color: #1890ff; color: white;">编辑
</button>
                    <button class="action-btn" style="background-color: #ff4d4f; color: white;">删除
</button>
                </td>
            `;
            tableBody.appendChild(row);
        });
}
function updateCharts(operationTypeStats, operationResultStats) {
        const operationTypeChart = window.operationTypeChart;
        const operationResultChart = window.operationResultChart;
        const operationTypeData = [
            { value: operationTypeStats.SystemUpdate, name: '系统更新', itemStyle: { color: '#1890ff' } },
            { value: operationTypeStats.DeviceMaintenance, name: '设备维护', itemStyle: { color: '#13c2c2' } },
            { value: operationTypeStats.DataBackup, name: '数据备份', itemStyle: { color: '#722ed1' } }
        ];
        operationTypeChart.setOption({
            series: [
                {
                    data: operationTypeData
                }
            ]
        });
```

```
    const operationResultData = [
        { value: operationResultStats.Success, itemStyle: { color: '#52c41a' } },
        { value: operationResultStats.Failure, itemStyle: { color: '#ff4d4f' } }
    ];
    operationResultChart.setOption({
        series: [
            {
                data: operationResultData
            }
        ]
    });
}
package handlers
import (
    "github.com/gogf/gf/frame/g"
    "github.com/gogf/gf/net/ghttp"
    "project/internal/models"
)
func MaintenanceLogHandler(r *ghttp.Request) {
    // 获取维护操作日志数据
    logs, err := models.GetMaintenanceLogs()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "获取维护操作日志数据失败",
        })
        return
    }
    // 获取操作类型统计信息
    operationTypeStats, err := models.GetOperationTypeStats()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "获取操作类型统计信息失败",
        })
        return
    }
    // 获取操作结果统计信息
    operationResultStats, err := models.GetOperationResultStats()
    if err != nil {
        r.Response.WriteJson(g.Map{
            "code": 500,
            "msg":  "获取操作结果统计信息失败",
        })
        return
    }
    r.Response.WriteJson(g.Map{
        "code":                 200,
        "msg":                  "成功",
        "logs":                 logs,
```

```
            "operationTypeStats":     operationTypeStats,
            "operationResultStats": operationResultStats,
        })
}
package models
import (
    "github.com/gogf/gf/database/gdb"
    "github.com/gogf/gf/frame/g"
)
// MaintenanceLog 维护操作日志结构体
type MaintenanceLog struct {
    OperationType string `json:"OperationType"`
    Operator      string `json:"Operator"`
    OperationTime string `json:"OperationTime"`
    OperationResult string `json:"OperationResult"`
    DeviceName    string `json:"DeviceName"`
    LogStatus     string `json:"LogStatus"`
}
// OperationTypeStats 操作类型统计信息结构体
type OperationTypeStats struct {
    SystemUpdate    int `json:"SystemUpdate"`
    DeviceMaintenance int `json:"DeviceMaintenance"`
    DataBackup      int `json:"DataBackup"`
}
// OperationResultStats 操作结果统计信息结构体
type OperationResultStats struct {
    Success int `json:"Success"`
    Failure int `json:"Failure"`
}
// GetMaintenanceLogs 获取维护操作日志数据
func GetMaintenanceLogs() ([]MaintenanceLog, error) {
    var logs []MaintenanceLog
    err := g.DB().Table("maintenance_logs").Order("operation_time DESC").Scan(&logs)
    return logs, err
}
// GetOperationTypeStats 获取操作类型统计信息
func GetOperationTypeStats() (OperationTypeStats, error) {
    var stats OperationTypeStats
    err := g.DB().Table("maintenance_logs").Fields("SUM(CASE WHEN operation_type = '系统更新' THEN 1
ELSE 0 END) AS system_update, SUM(CASE WHEN operation_type = '设备维护' THEN 1 ELSE 0 END) AS
device_maintenance, SUM(CASE WHEN operation_type = '数据备份' THEN 1 ELSE 0 END) AS
data_backup").One().Scan(&stats)
    return stats, err
}
```